

VIETNAM NATIONAL UNIVERSITY OF HOCHIMINH CITY
THE INTERNATIONAL UNIVERSITY
SCHOOL OF COMPUTER SCIENCE AND ENGINEERING



A FASHION RECOMMENDATION SYSTEM VIA SEQUENCE MINING FROM SOCIAL MEDIA

By

Hoàng Văn Mạnh

A thesis proposal submitted to the School of Computer Science and Engineering

In partial fulfillment of the requirements for the degree of

Bachelor of Data Science

Ho Chi Minh City, Viet Nam

2025-2026

A FASHION RECOMMENDATION SYSTEM VIA SEQUENCE MINING FROM SOCIAL MEDIA



SUPERVISED BY:

_____,
Dr. Nguyễn Thị Thanh Sang

TABLE OF CONTENTS

TABLE OF CONTENTS	2
ACKNOWLEDGEMENT	3
LIST OF FIGURES	3
LIST OF TABLES	3
ABBREVIATION	4
ABSTRACT	5
CHAPTER 1: INTRODUCTION	6
1.1. Background	6
1.2. Problem statement	7
1.3. Scope and Objectives	7
1.4. Assumptions and Solution	8
1.5. Structure of Thesis	9
CHAPTER 2: LITERATURE REVIEW	10
2.1. Overview of Social Media Recommendation System	10
2.2. Comment Recommendation Models on Social Platforms	12
2.3. Sequence Mining Techniques and Bi-LSTM in Recommendation System	15
2.4. Limitations and Research Gap	17
CHAPTER 3: METHODOLOGY	19
3.1. General Idea and Architecture	19
3.2. Data Collection and Description	20
3.3. Data Preprocessing and Exploratory Data Analysis	22
3.4. Sequence Mining Module	29
3.5. Bi-LSTM Model	30
3.6. Integration with Fashion Recommendation System	31
3.7. Implementation platform and libraries	32
CHAPTER 4: PRELIMINARY IMPLEMENTATION & RESULT	33
4.1. Sequence Mining Results	33
4.2. Bi-LSTM Performance	33
CHAPTER 5: CONCLUSIONS	34
REFERENCES	36
APPENDICES	28

ACKNOWLEDGEMENT

This thesis marks the culmination of my journey toward the Bachelor's degree in Data Science at the School of Computer Science and Engineering.

I would like to express my deepest gratitude to my supervisor, **Dr. Nguyễn Thị Thanh Sang**, for her invaluable guidance and unwavering support throughout this semester. She generously devoted her time to review my progress and provided insightful feedback whenever I faced challenges. This thesis would not have been possible without her dedicated mentorship.

I am also sincerely thankful to my friends, who have been by my side from the very beginning—sharing coding sessions, exam stress, and countless academic hurdles. Their encouragement and companionship have been a source of motivation and balance throughout this journey. The memories we created together will always be treasured.

Completing this thesis has been a deeply rewarding experience that reflects the knowledge, perseverance, and growth I have gained over the past four years.

LIST OF TABLES

Table 3.2.1: Description of the Post dataframe

Table 3.2.2: Description of the Comment dataframe



LIST OF FIGURES

Figure 3.2.1: Post DataFrame

Figure 3.2.2: Comment DataFrame

Figure 3.3.1 Missing value before and after handling.

Figure 3.3.2 Missing value before and after handling.

Figure 3.3.3: Likes and Comments Distribution

Figure 3.3.4: Distribution of Comments per Post

Figure 3.3.5: Posting Frequency by Month

Figure 3.3.6: Average response time between post and first comment

Figure 3.3.7: Word Cloud of Post Keywords

Figure 3.3.8: Word Cloud of Comment Keywords

Figure 3.3.9: Distribution of Comment Sentiments

ABBREVIATION

Notation	Description
UGC	User-generated content
TF-IDF	Term Frequency - Inverse Document Frequency
RS	Recommendation System
MC	Markov Chain
LSTM	Long Short-Term Memory
RNN	Recurrent Neural Network
FFNs	Feed-Forward Neural Networks
BiLSTM	Bidirectional Long Short-Term Memory
CNN	Convolutional Neural Network
CBF	Content-based Filtering
CF	Collaborative Filtering
MHA	Multi-head Attention
SASRec	Self-Attentive Sequential Recommendation
MLM	Masked Language Modeling
JSON	JavaScript Object Notation
CSV	Comma Separated Values
SA	Sentiment Analysis
DaViT	Dual Attention Vision Transformer

ViT	Vision Transformer
BERT	Bidirectional Encoder Representations from Transformers



ABSTRACT

The proliferation of social media has generated vast amounts of user-generated content, serving as a real-time indicator of fashion trends. However, conventional recommendation systems often face challenges in capturing the **dynamic evolution** of user preferences over time and fail to fully leverage unstructured multimodal data. This thesis addresses these limitations by proposing a novel recommendation framework that integrates sequence mining with multimodal feature learning.

Technically, **I developed** a hybrid deep learning architecture based on **Bidirectional Long Short-Term Memory (BiLSTM)** networks coupled with a **Multi-Head Attention** mechanism. This combination allows the model to effectively learn long-term temporal dependencies while assigning varying attention weights to critical interactions within a user's history. To further enhance representation learning and mitigate data sparsity, the system fuses visual features from images with semantic attributes derived from captions and hashtags.

Experimental evaluations on a dataset collected from Instagram demonstrate that the proposed method **outperforms** standard baseline models in terms of ranking quality and recommendation accuracy. These findings confirm that modeling sequential behavior and incorporating multimodal context significantly improves the personalization of fashion recommendations.

CHAPTER 1: INTRODUCTION

1.1. Background:

1.1.1. The Digital Transformation of Fashion Retail and Social Commerce

The fashion industry has undergone a paradigm shift from traditional brick-and-mortar retailing to digital-first ecosystems. In recent years, this transformation has been further accelerated by the rise of **Social Commerce**—a convergence of social media and e-commerce. Platforms such as **Instagram**, **Pinterest**, and **TikTok** have evolved beyond mere communication tools to become primary drivers of fashion trends and consumption behaviors.

Unlike traditional retail, where trends were dictated top-down by magazines and designers, modern fashion trends are increasingly driven bottom-up by user-generated content (UGC). Influencers and everyday users actively create and share content, effectively becoming tastemakers. This shift implies that the "signal" for what is fashionable or desirable is no longer static but is continuously generated in real-time within social networks [1].

1.1.2. Characteristics of Social Media Data

The interaction between users and fashion content on social platforms generates a massive volume of data, characterized by its **high dimensionality** and **heterogeneity**. This data is not limited to simple transactional records (e.g., "User A bought Item B") but encompasses complex, unstructured formats:

- **Visual Content:** Fashion is inherently visual. Images and videos provide critical information about style, color, texture, and visual aesthetics that text alone cannot capture. [2]
- **Textual Context:** Captions, hashtags, and comments provide semantic context, describing the occasion, mood, or specific brand attributes associated with an image.
- **Temporal Dynamics:** User interactions occur in sequences. A user's preference today is influenced by their recent history and broader seasonal trends.

As highlighted in a systematic review by Albalawi *et al.* [3], applying data mining techniques to this rich, multimodal social data allows for the extraction of **latent behavioral patterns**, which are inaccessible through traditional transaction logs.

1.1.3. Information Overload and the Need for Personalization

While the abundance of online fashion content offers users unprecedented choice, it simultaneously creates a significant cognitive burden known as **information overload** [4]. In a typical online fashion store or social feed, a user is exposed to thousands of items daily. Without effective filtering mechanisms, the process of discovering relevant products becomes tedious and inefficient.

In this context, **Recommendation Systems (RS)** have transitioned from being optional value-added features to essential components of the digital infrastructure. A robust RS serves a dual purpose:

1. **For the User:** It reduces search costs and improves decision-making by curating a personalized selection of items that align with their evolving style.
2. **For the Platform:** It increases user engagement, retention, and conversion rates by ensuring that users are continuously presented with content that resonates with their interests.

Therefore, developing an intelligent system capable of understanding the sequential and multimodal nature of social media data is critical for solving the information overload problem in the fashion domain.

1.2. Problem statement

Despite the widespread adoption of Traditional **Recommendation Systems** (RS) such as **Collaborative Filtering** (CF) and **Matrix Factorization** (MF), applying these methods to the social media fashion domain presents significant challenges that remain unaddressed.

First, the inability to capture dynamic temporal preferences. Fashion is an inherently time-sensitive domain, characterized by rapidly evolving trends and seasonal changes. Traditional CF methods typically treat user historical interactions as a static set, ignoring the sequential order of events. For instance, a user's interest in "winter coats" in December should not influence recommendations in July. Static models fail to model these **sequential dependencies** and short-term interest drifts, leading to recommendations that are outdated or contextually irrelevant [5].

Second, the underutilization of multimodal content. Most existing systems rely heavily on User-ID and Item-ID matching. However, in the context of social media, user engagement is primarily driven by visual aesthetics (e.g., style, color, texture) and semantic context (e.g., hashtags, captions). Purely ID-based approaches suffer from the **"Cold-Start" problem**: they cannot recommend new items that have no prior interaction history because they lack the ability to "see" or "understand" the content of the image itself [6].

Third, the challenge of Data Sparsity. Social media datasets are typically large-scale but extremely sparse. A single user interacts with only a minuscule fraction of the millions of available fashion items (often less than 0.1%). Conventional Matrix Factorization models struggle to learn high-quality latent vectors from such sparse interaction matrices, resulting in poor prediction accuracy and a tendency to recommend only popular items (popularity bias) rather than personalized niche products [7].

Conclusion: Consequently, there is a critical need for a unified framework that can simultaneously: (1) model the sequential evolution of user tastes using deep learning, and (2) leverage rich multimodal data (visual and textual) to enhance representation learning and mitigate data sparsity.

1.3. Scope and Objectives

1.3.1. Scope

The boundaries of this research are defined as follows:

- **Data Scope:** The study utilizes a dataset collected from social media platforms (specifically Instagram), comprising user-item interactions, image data, and

textual metadata (captions, hashtags). The focus is strictly on **implicit feedback** (likes, comments) rather than explicit ratings (1-5 stars).

- **Domain Scope:** The application domain is restricted to **Fashion and Apparel** (clothing, accessories, footwear). Other categories found on social media are excluded to ensure domain-specific optimization.
- **Methodological Scope:** The research focuses on the **Next-Item Prediction** task within the context of Sequential Recommendation. The core contribution lies in the algorithmic improvement of the ranking model.

limitations (Out of Scope):

- **Full-Stack Implementation:** The thesis focuses on the backend model and algorithmic performance. The development of a full-fledged user interface (mobile/web app) or a commercial deployment pipeline is outside the scope of this work.
- **Real-time Latency:** While efficiency is considered, extreme optimization for millisecond-level real-time inference is not the primary focus of this academic study.

1.3.2. Objectives

The primary goal of this thesis is to design, implement, and evaluate a Sequential Recommender System specifically tailored for the fashion domain on social media. The system

aims to predict the next fashion item a user is likely to interact with by analyzing their historical interaction sequence.

To achieve this goal, the research focuses on the following specific objectives:

- **Architecture Design:** To propose a hybrid Deep Learning architecture that integrates Bidirectional LSTM (BiLSTM) for capturing sequential dependencies and Multi-Head Attention mechanisms for prioritizing critical historical interactions.
- **Multimodal Feature Fusion:** To develop a pipeline for extracting and fusing heterogeneous data features, specifically combining **visual embeddings** (via advanced Computer Vision backbones) with **semantic attributes** (from captions and hashtags) to address the data sparsity problem.
- **Empirical Evaluation:** To conduct comprehensive experiments comparing the proposed model against established baselines (standard LSTM, BiLSTM) using standard ranking metrics such as Hit Rate, Mean Reciprocal Rank (MRR), and Normalized Discounted Cumulative Gain (NDCG).

1.4. Assumptions and Solution

1.4.1. Assumptions

To ensure the feasibility and validity of this study, the following assumptions are established:

- **Assumption 1: Reliability of Filtered Implicit Feedback.** ... Specifically, only comments classified as **Neutral or Positive** by a context-aware Deep Learning model (e.g., **Twitter-RoBERTa**) are treated as valid signals. We assume that a robustly

pre-trained Transformer model can effectively distinguish between genuine interest and noisy/negative remarks better than rule-based heuristics.

- **Assumption 2: Existence of Sequential Patterns.** We premise that fashion choices are not random isolated events but follow learnable sequential dependencies (e.g., seasonal transitions or style evolution). A user's past interactions contain sufficient signal to predict their future behavior.
- **Assumption 3: Multimodal Correlation.** We assume that visual features (image content) and semantic features (textual context) are complementary. Combining these modalities provides a richer representation of an item than using either modality in isolation, particularly for overcoming the cold-start problem.

1.4.2. Solution

To address the limitations of existing methods identified in the Problem Statement, this thesis proposes a **Multimodal Sequential Recommendation Framework**. The solution architecture consists of three core components:

- **Multimodal Feature Extraction Module:**
 - Unlike traditional ID-based approaches, the system utilizes pre-trained deep learning backbones to extract rich feature vectors.
 - **Visual Encoder:** Utilizes advanced Computer Vision models (Florence-2) to capture high-level aesthetic features from fashion images.
 - **Textual Encoder:** Employs NLP techniques (BERT-based models) to process captions and hashtags into semantic embeddings.
- **Sequential Learning Module (BiLSTM):**

- The core of the recommender is built upon a **Bidirectional Long Short-Term Memory (BiLSTM)** network.
- This component processes the sequence of user interactions in both forward and backward directions, enabling the model to capture complex temporal dynamics and the evolution of user style over time.
- **Attention Mechanism (Multi-Head Attention):**
 - To enhance prediction accuracy, a Multi-Head Attention layer is integrated on top of the BiLSTM.
 - This mechanism allows the model to dynamically assign importance weights to different historical interactions, filtering out noise and focusing on the most relevant past items that influence the current prediction.

The final output of the system is a ranked list of items that maximizes the probability of user interaction in the next time step.

1.5. Structure of Thesis

This is the flow of the thesis. This part briefly indicates how each component of the report contributes to the acknowledgement of the readers:

Introduction: Outlines the research context, problem statement, objectives, scope, and the proposed assumptions and solution.

Literature Review: Reviews fundamental concepts of Recommender Systems and analyzes related works on Sequence Mining, BiLSTM networks, and Attention mechanisms to establish the theoretical basis.

Methodology: Details the proposed **Multimodal Sequential Recommendation Framework**, focusing on the system architecture, multimodal feature extraction pipeline, and the BiLSTM-Attention modeling process.

Implementation and Results: Describes the experimental setup, dataset characteristics, and evaluation metrics. It presents a comparative analysis against baseline models to validate the effectiveness of the proposed approach.

Conclusion and Future Work: Summarizes the key contributions and limitations of the study, while suggesting potential directions for future research and optimization.

References: Lists all academic sources, journals, conference papers, and datasets cited throughout the thesis, formatted according to IEEE standards.

CHAPTER 2: LITERATURE REVIEW

2.1. Overview of Recommendation Systems

2.1.1. Introduction and Formal Definition

A Recommender System (RS) is a subclass of information filtering systems that seeks to predict the "rating" or "preference" a user would give to an item [x]. In the era of big data, RS plays a pivotal role in mitigating the **information overload** problem by filtering vital information out of a massive amount of dynamically generated data.

Formally, the recommendation problem can be formulated as follows: Let $U = \{u_1, u_2, \dots, u_m\}$ be the set of users and $I = \{i_1, i_2, \dots, i_n\}$ be the set of items. The utility function f measures the suitability of item i to user u , denoted as:

$$f: U \times I \rightarrow R$$

where R is a totally ordered set representing the user's feedback (e.g., a rating score or purchase probability). The objective of an RS is to estimate the utility function f for the user-item pairs (u, i) that have not yet been observed, and subsequently recommend items $\hat{i}$ that maximize this utility for each user:

$$\forall u \in U, \hat{i}_u = \arg \max_{i \in I} f(u, i)$$

Table 2.1 Notations for the Recommendation Scoring Function

Notation	Definition
U	The set of all users
I	The set of all items
u	A specific user, where $u \in U$
i	A specific item, where $i \in I$
$\hat{i}_u$	The recommended item for user u , predicted by the model.
$f(u, i)$	The scoring function measuring the relevance of item i to user u
$\text{Argmax}_{i \in I}$	The operator that returns the item i that maximizes the value of a function
$\forall u \in U$	“For all users in U ” — the rule applies to every user

2.1.2. Types of User Feedback

Data used in RS is generally categorized into two types:

- **Explicit Feedback:** Direct input from users regarding their interest in items (e.g., 1-5 star ratings on Amazon, Shopee). While precise, this data is often scarce and hard to collect.
- **Implicit Feedback:** Inferred from user behavior (e.g., clicks, page views, purchase history, dwell time). In social media contexts (like Instagram), implicit feedback (likes, comments) is abundant but inherently noisy. This thesis primarily focuses on **implicit feedback** due to the nature of the dataset.

2.1.3. Classification of Recommendation Approaches

According to the taxonomy proposed by Burke [x], recommendation techniques are generally categorized into five main classes based on their data sources and algorithmic logic:

A. Content-Based Filtering (CBF):

Mechanism: Content-based filtering operates on the principle that if a user liked an item in the past, they will like similar items in the future. This approach relies heavily on **feature extraction**. Items are represented as vectors of attributes (e.g., keywords, categories, or visual descriptors), and user profiles are constructed by aggregating the features of items the user has previously interacted with. The relevance of a candidate item is typically determined by calculating the **Cosine Similarity** between the user profile vector and the item vector. In textual domains, techniques such as **TF-IDF** (Term Frequency-Inverse Document Frequency) are commonly employed to weigh feature importance.

- **Advantages:**

- **User Independence:** The model relies solely on the active user's history, eliminating the need for data from other users.
- **Item Cold-Start Mitigation:** New items can be recommended immediately upon insertion, provided their attributes are populated.
- **Limitations:**
 - **Overspecialization:** CBF systems tend to recommend items that are too similar to the user's existing portfolio, creating a "filter bubble" and lacking **serendipity** (the ability to surprise the user with novel discoveries).\
 - **Feature Engineering Bottleneck:** The effectiveness of the model is strictly bound by the quality of the item descriptions. In the fashion domain, manually tagging attributes (e.g., "vintage," "bohemian") is labor-intensive and subjective, necessitating the use of automated deep learning feature extractors.

B. Collaborative Filtering (CF):

Mechanism: Collaborative Filtering is widely regarded as the most successful approach in traditional recommender systems. It operates on the assumption that users who agreed in the past will agree in the future. CF is generally divided into two sub-categories:

- **Memory-Based (Neighborhood Models):** These algorithms calculate similarity directly on the interaction matrix.
 - + **User-Based:** Identifies a peer group ("neighbors") similar to the target user and recommends items liked by this group.

- + **Item-Based:** Identifies items that are frequently consumed together by the user base. Similarity is often measured using **Pearson Correlation** or **Jaccard Similarity**.
- **Model-Based (Matrix Factorization):** These techniques, such as **Singular Value Decomposition (SVD)**, decompose the high-dimensional user-item interaction matrix into lower-dimensional **latent vectors**. The dot product of a user vector and an item vector predicts the preference score.
- **Advantages:**
 - **Serendipity:** CF can identify latent patterns and recommend items from entirely different categories, provided that similar users have engaged with them.
 - **Domain Independence:** It does not require domain-specific knowledge or metadata interpretation.
- **Limitations:**
 - **Cold-Start Problem:** The system fails to handle new users or new items that lack interaction history.
 - **Data Sparsity:** In social media contexts, the interaction matrix is extremely sparse (often >99% empty), making it difficult to find reliable neighbors or train robust latent factors [x].

C. Demographic-Based Filtering

Mechanism: This approach classifies users based on personal attributes such as age, gender, occupation, and geographic location. Recommendations are generated by identifying the

aggregate preferences of a specific demographic cluster. For instance, a system might recommend "formal office wear" to users identified as corporate professionals in a specific age bracket.

- **Advantages:**

- **New User Cold-Start Solution:** Unlike CF, this method can generate recommendations immediately after a user registers and provides basic profile information, without requiring a history of interactions.

- **Limitations:**

- **Coarse Granularity:** Demographic profiles are often too broad to capture the nuances of individual taste, particularly in subjective domains like fashion. Two users of the same age and location may have diametrically opposed aesthetic preferences.
- **Privacy Concerns:** The collection of sensitive personal data raises significant privacy issues and compliance challenges (e.g., GDPR).

D. Knowledge-Based Filtering

Mechanism: Knowledge-based systems utilize explicit domain knowledge regarding how specific item features satisfy user needs. They are typically employed in domains where items are complex, expensive, and purchased infrequently (e.g., real estate, automobiles, or electronics).

- *Constraint-Based:* Users specify a set of requirements (constraints), and the system filters items that violate these rules.
- *Case-Based Reasoning (CBR):* The system retrieves similar "cases" from the past to solve the current user's problem.

- **Advantages:**

- **Precision:** Guaranteed to meet the explicit constraints set by the user.
- **No Cold-Start:** Does not rely on historical interaction logs.
- **Limitations:**
 - **Scalability and Cost:** Developing the knowledge base requires significant manual effort from domain experts (knowledge acquisition bottleneck). It is ill-suited for the dynamic, unstructured data found on social media platforms.

E. Hybrid Systems

Mechanism: Hybrid systems integrate two or more of the aforementioned techniques to leverage their complementary strengths. Common hybridization strategies include:

- *Weighted:* Aggregating the scores of different recommendation components numerically.
- *Switching:* Selecting a specific recommendation component based on the context (e.g., using Demographic filtering for a new user, then switching to CF once sufficient data is collected).
- *Feature Augmentation:* Using the output of one model as a feature input for another.

Relevance to Thesis: Current state-of-the-art research often employs **Deep Learning-based Hybrid** approaches. For instance, this thesis proposes a system that is conceptually hybrid: it utilizes **Content-Based** principles (via visual and textual embedding) to understand item characteristics, integrated within a **Sequential** (collaborative) framework to model user behavior over time.

2.2. Sequential Recommender Systems - SRS

Unlike traditional Collaborative Filtering methods, which model user preferences based on a static set of historical interactions, **Sequential Recommender Systems (SRS)** treat user behavior as a dynamic sequence of events. The core premise of SRS is that the order of interactions is significant—a user's next action is conditionally dependent on their recent behaviors and the temporal evolution of their interests.

Problem Formulation: Let $S_u = \{i_1, i_2, \dots, i_t\}$ denote the sequence of items interacted with by user u , sorted chronologically. The objective of an R is to predict the next item i_{t+1} from the item set I by maximizing the conditional probability:

$$\arg \max_{i \in I} P(i_{t+1} | S_u)$$

Table 2.2 Notations for the Sequential Recommender System Function

Notation	Definition
$S_u = \{i_1, i_2, \dots, i_t\}$	The historical sequence of items interacted with by user u , sorted chronologically
I	The set of all candidate items in the system
i_{t+1}	The target item to be predicted at the next time step

2.2.1. Early Approaches: Markov Chains (MC)

Initial efforts to model sequential dynamics relied on **Markov Chain** theory. The core assumption is the Markov Property, which posits that the next state depends solely on the current state (or a finite window of previous states).

Rendle *et al.* [x] advanced this field by proposing **Factorized Personalized Markov Chains (FPMC)**. FPMC hybridizes Matrix Factorization with first-order Markov Chains. However, MC-based models inherently suffer from the "**memoryless**" property. As the order of the chain increases to capture longer dependencies, the state space grows exponentially, rendering the model computationally infeasible for sparse datasets.

2.2.2. Recurrent Neural Networks (RNN)

To overcome the limited memory capacity of Markov Chains, **Recurrent Neural Networks (RNNs)** were introduced to the recommendation domain. Unlike standard Feed-Forward Neural Networks (FNNs) which assume inputs are independent of each other, RNNs are designed specifically for sequential data. They operate on the principle that the output at the current time step depends not only on the current input but also on the "memory" of previous inputs.

This memory is maintained via a **hidden state** vector, which loops back into the network, allowing information to persist as the model traverses the sequence. This architecture makes RNNs theoretically capable of capturing dependencies of arbitrary length.

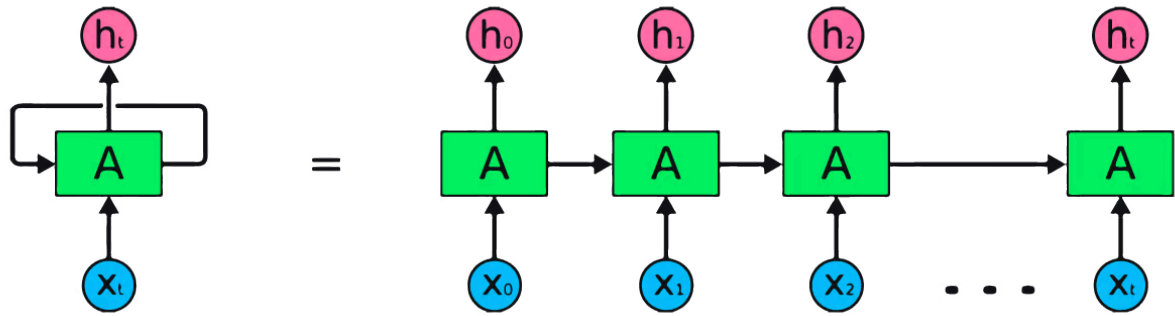


Figure 2.1 Vanilla Recurrent Neural Network (RNN) Architecture

Mathematical Formulation: Formally, let x_t be the input vector at time step t . The RNN computes the hidden state h_t and the output y_t using the following recurrence relation:

- **Hidden state update:**

$$h_t = f(W_h h_{t-1} + W_x x_t + b_h)$$

- **Output:**

$$y_t = g(W_y h_t + b_y)$$

Table 2.2.1 Notation for the Vanilla RNN Formulation

Notation	Definition
x_t	Input vector at time t
h_t	Hidden state vector at time t (current memory)
h_{t-1}	Hidden state vector at time $t-1$ (previous memory)
y_t	Output vector at time t (predicted probability distribution)
W_h	Weight matrix for hidden-to-hidden recurrent connection
W_x	Weight matrix for input-to-hidden connection
W_y	Weight matrix for hidden-to-output connection
b_h, b_y	Bias vectors for hidden state and output

$f(.)$	Non-linear activation function (typically <i>tanh</i> or <i>ReLU</i>)
$g(.)$	Output activation function (typically <i>Softmax</i> for classification)

Seminal Work: Hidasi *et al.* [x] pioneered this approach with **GRU4Rec**, applying Gated Recurrent Units to session-based data.

Critique: However, standard RNNs encounter the **Vanishing Gradient Problem** during Backpropagation Through Time (BPTT). As the sequence length increases, gradients tend to approach zero, inhibiting the learning of dependencies from interactions that occurred far in the past.

2.2.3. Long Short-Term Memory (LSTM)

Architectural Fundamentals Standard Recurrent Neural Networks often falter when tasked with learning dependencies that bridge long temporal gaps, primarily due to the **Vanishing Gradient Problem**. As gradients propagate backward through numerous time steps, they tend to decay exponentially, rendering the weights of early layers effectively untrainable. To address this pathology, Hochreiter and Schmidhuber [x] proposed the **Long Short-Term Memory (LSTM)** architecture.

The distinct innovation of the LSTM is the introduction of a **Memory Cell (c_t)**, often described as a "constant error carousel." Unlike the hidden state in standard RNNs which is overwritten at every step, the cell state allows information to flow along the sequence chain with only minor linear interactions. This preservation of error signals is regulated by specialized **gating mechanisms**, which act as differentiable filters determining the extent to which the memory cell should be retained, updated, or exposed.

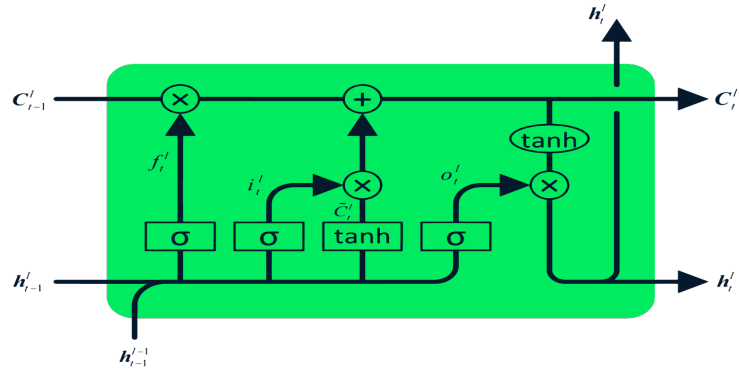


Figure 2.2 Long Short-Term Memory (LSTM) Cell Architecture

Mathematical Derivation For a given input vector x_t and the preceding hidden state h_{t-1} , the LSTM transition functions are formally derived as follows.

- **Gating Logic:** The network first computes the activation vectors for the **Forget**, **Input**, and **Output** gates. The logistic sigmoid function (σ) is utilized here to output values in the range (0, 1), effectively acting as a "soft" switch.

- **Forget Gate:**

$$f_t = \sigma (W_f x_t + U_f h_{t-1} + b_f)$$

- **Input Gate:**

$$i_t = \sigma (W_i x_t + U_i h_{t-1} + b_i)$$

- **Output Gate:**

$$o_t = \sigma (W_o x_t + U_o h_{t-1} + b_o)$$

- **Memory and State Updates:** The core update logic involves calculating a candidate memory vector ($\check{c}_t$) using the hyperbolic tangent function. The actual cell state c_t is then updated via linear interpolation between the old state and the new candidate, governed by the forget and input gates. Finally, the hidden state h_t is derived by filtering the cell state through the output gate.

- **Candidate cell**

$$\check{C}_t = \tanh(W_C x_t + U_C h_{t-1} + b_C)$$

- **Update cell state**

$$C_t = f_t \odot C_{t-1} + i_t \odot \check{C}_t$$

- **Hidden state**

$$h_t = o_t \odot \tanh(C_t)$$

Table 2.2.2 Notation for the LSTM Formulation

Notation	Definition
x_t	Input vector at time step t
h_t	Hidden state at time t
h_{t-1}	Previous hidden state
C_t	Cell state at time t
C_{t-1}	Previous cell state
f_t	Forget gate activation
i_t	Input gate activation

o_t	Output gate activation
$\check{C}_t$	Candidate cell state
$W_f W_b W_o W_c$	Weight matrices for input $\rightarrow$ gates
$U_f U_b U_o U_c$	Weight matrices for hidden state $\rightarrow$ gates
$b_f b_b b_o b_c$	Bias terms for each gate
$\sigma(\cdot)$	Sigmoid activation function
$\tanh(\cdot)$	Hyperbolic tangent activation function
$\odot$	Element-wise multiplication
t	Time step index

Implications for Fashion Recommendation In the context of this thesis, the architectural separation of c_t and h_t provides a significant theoretical advantage. The **Cell State (c_t)** is hypothesized to capture a user's intrinsic, evolving style, while the **Hidden State (h_t)** captures **transient shopping intent**. This mechanism allows the proposed model to generate recommendations that are contextually relevant to the immediate session without losing sight of the user's long-term preferences.

2.2.4. Bidirectional Long Short-Term Memory (BiLSTM)

While standard LSTMs have proven effective in capturing long-term dependencies, they process sequences in a strictly unidirectional manner (chronologically from t_1 to t_n). This imposes a limitation: the representation of the user's state at time t is derived solely from past interactions, ignoring the context of future interactions available in the historical training data.

To address this, the **Bidirectional LSTM (BiLSTM)** extends the architecture by training two separate LSTM layers on the input sequence:

- **Forward Layer:** Processes the sequence from start to end ($\mathbf{x}_1 \rightarrow \mathbf{x}_T$)
- **Backward Layer:** Processes the sequence in reverse order ($\mathbf{x}_T \rightarrow \mathbf{x}_1$)

By concatenating the hidden states from both directions, the BiLSTM generates a comprehensive representation that encapsulates information from both the past and the future context surrounding a specific time step.

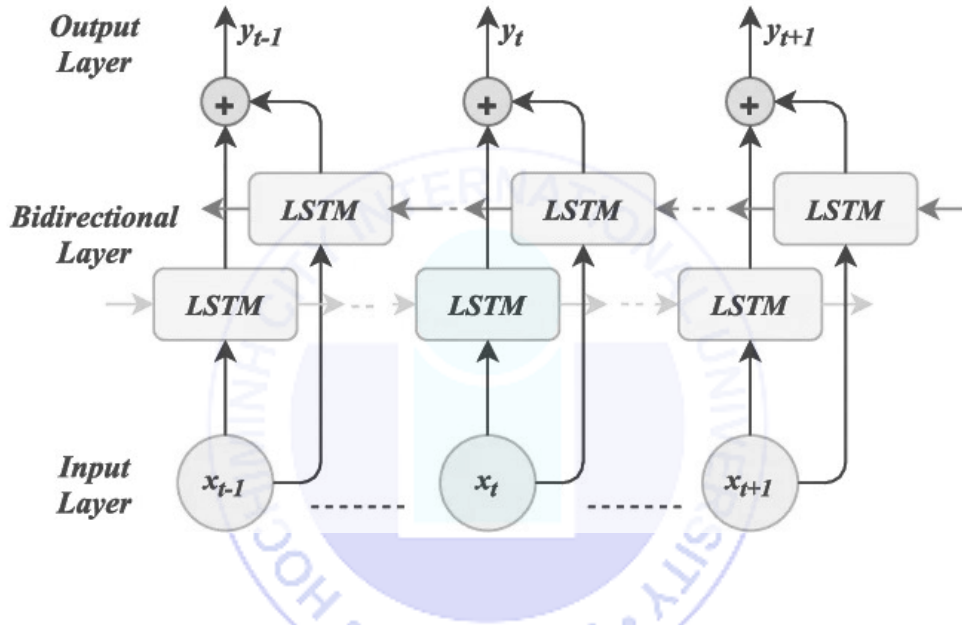


Figure 2.2 Bidirectional Long Short-Term Memory (BiLSTM) Architecture

Mathematical Formulation: Let $\mathbf{x}_t$ be the input embedding at time t . The BiLSTM computes the forward hidden state $\vec{h}$ and the backward hidden state $\overleftarrow{h}$ using the standard LSTM transition functions (as defined in [Section 2.2.3](#)):

- **Forward LSTM:**

$$\vec{h}_t = LSTM_{forward}(\mathbf{x}_t, \vec{h}_{t-1})$$

- **Backward LSTM:**

$$\overleftarrow{h}_t = LSTM_{backward}(x_t, \overleftarrow{h}_{t+1})$$

The final latent representation y_t for the item at time step t is formed by concatenating these two vectors:

$$y_t = [\overrightarrow{h}_t \oplus \overleftarrow{h}_t]$$

For sequence-level classification or next-item prediction, the final output is often a transformation of the concatenated states, passed through a dense layer:

$$\hat{y} = softmax(W_{out} \bullet y_t + b_{out})$$

Table 2.2.3 Notation for the BiLSTM Formulation

Notation	Definition
$\overrightarrow{h}_t$	Forward Hidden State. Captures context from the past (t-1)
$\overleftarrow{h}_t$	Backward Hidden State. Captures context from the "future" (t+1) relative to the current step in the sequence log
$LSTM_{forward}$	The forward processing LSTM unit
$LSTM_{backward}$	The backward processing LSTM unit
$\oplus$	Concatenation Operator. Joins two vectors end-to-end
y_t	Combined Hidden State. The final bidirectional representation of the input at time t
W_{out}, b_{out}	Weight matrix and bias for the final projection layer

Justification for Adoption (Relevance to Thesis): The adoption of BiLSTM in this research is predicated on two strategic advantages relevant to social media mining:

- **Contextual Enrichment (Training Phase):** Although real-time recommendation is unidirectional, during the *offline training phase* on historical logs, the "future" interactions (items clicked *after* item i) provide critical context for understanding what item i actually represents to the user. BiLSTM leverages this non-causal information to refine the feature embeddings.
- **Noise Robustness:** User interaction logs on platforms like Instagram are inherently noisy (e.g., accidental clicks or random browsing). By aggregating context from two opposing temporal directions, the model can better differentiate between a consistent preference trend (which appears in both directions) and random noise (which often lacks bidirectional coherence).

2.3. Attention Mechanisms in Deep Learning

While Recurrent Neural Networks (RNN/LSTM/BiLSTM) have established a baseline for sequential modeling, they suffer from two fundamental limitations:

1. **Information Bottleneck:** The network must compress the entire sequential context into a fixed-size hidden vector $\mathbf{h}_t$, leading to information loss in long sequences [x].
2. **Sequential Computation:** RNNs require the previous hidden state $\mathbf{h}_{t-1}$ to compute $\mathbf{h}_t$, preventing parallelization during training.

To circumvent these issues, the **Attention Mechanism** was introduced, marking a paradigm shift in Deep Learning.

Before detailing the algorithmic mechanics, **Table 2.3** summarizes the mathematical notations utilized in this section.

Table 2.3 Mathematical Notations for the Attention Mechanisms

Notation	Definition
Q	Query matrix represents the current intent/focus
K	Key matrix represents the identifying features of past items
V	Value Matrix represents the semantic content to be retrieved
d_k	Dimensionality of Queries and Keys
$\sqrt{d_k}$	Scaling factor to prevent gradient saturation
h	Number of Attention Heads
W^Q, W^K, W^V	Learnable linear projection matrices
α_{ij}	Attention weight (relevance of item j to item i)

2.3.1. Scaled Dot-Product Attention

The fundamental computational unit employed in this framework is **Scaled Dot-Product Attention**, proposed by Vaswani *et al.* [x]. Unlike additive attention mechanisms [x] which use a feed-forward network to compute compatibility, Dot-Product attention is computationally faster and more space-efficient due to optimized matrix multiplication implementations.

Mathematical Formulation: Conceptually, the mechanism maps a **Query (Q)** to a set of **Key (K) - Value (V)** pairs. The output is computed as a weighted sum of the values, where the weight assigned to each value is determined by the compatibility function of the query with the corresponding key.

Formally, given the input matrices Q, K, V , the attention matrix is computed as follows:

$$Attention(Q, K, V) = softmax\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Algorithmic Analysis: The computation involves four sequential steps:

1. **Similarity Measurement (QK^T):** The dot product between Query and Key calculates the alignment score. A high dot product indicates that the historical interaction (Key) is highly relevant to the current intent (Query).
2. **Scaling ($\frac{1}{\sqrt{d_k}}$):** The scores are divided by $\sqrt{d_k}$ *justification:* In high-dimensional spaces, dot products can grow large, pushing the Softmax function into saturation regions (extremely small gradients). Scaling stabilizes the gradients during training, preventing the **Vanishing Gradient Problem**.
3. **Normalization (Softmax):** Converts scores into probabilities ($\sum \alpha = 1$)
4. **Weighted Aggregation (V):** Sums the Values weighted by attention scores, effectively filtering out noise.

2.3.2. Self-Attention Mechanism

Unlike standard encoder-decoder attention used in machine translation (where Q comes from the decoder and K, V from the encoder), this thesis employs **Self-Attention** (or Intra-Attention).

Definition: In Self-Attention, the Queries, Keys, and Values are all derived from the **same source** in this case, the hidden states of the **user interaction sequence**.

$$Q = XW^Q; K = XW^K; V = XW^V$$

Relevance to Recommendation (SASRec): As demonstrated by Kang and McAuley in the seminal SASRec (Self-Attentive Sequential Recommendation) [x], Self-Attention is superior for modeling user behavior because it models the correlation between *every pair* of items in the history directly. This allows the system to determine, for instance, that the purchase of a "shirt" at t_1 is highly relevant to the purchase of trousers at t_{50} , without the signal decaying through 49 intermediate steps as in an RNN.

2.3.3. Multi-Head Attention

Fashion preferences are multifaceted; a user might be interested in an item due to its **color** (visual), while simultaneously being interested in another due to its **brand** (semantic). A single attention head cannot capture these diverse representation subspaces simultaneously.

To address this, I employ **Multi-Head Attention**, which runs h parallel attention layers (heads). Each head projects the input into a different subspace:

$$head_i = Attention(QW_i^Q, KW_i^K, VW_i^V)$$

The outputs of all heads are concatenated and projected linearly to produce the final representation:

$$MultiHead(Q, K, V) = Concat(head_1, ..., head_h) W^O$$

This architectural choice ensures that the model provides a robust representation of user intent, capturing nuances that a single-vector representation would miss.

2.3.4. Integration with BiLSTM (Hybrid Architecture)

It is crucial to distinguish the application of Attention in this thesis from pure Transformer models.

- **Pure Transformers** rely on Positional Encodings to understand order, which can be sensitive to hyperparameter tuning for sparse social media sequences.
- **Our Hybrid Approach:** I utilize **BiLSTM** (Section 2.2.4) as the primary encoder to capture the complex sequential dynamics and temporal order. The **Multi-Head Attention** layer is applied *on top* of the BiLSTM outputs.

Mechanism of Synergy: The hidden states generated by the BiLSTM (H_{BiLSTM}) serve as the input X for the Attention layer. In this configuration:

- **BiLSTM** ensures **contextual completeness** and models the chronological evolution of style
- **Attention** acts as a **feature refinement and denoising layer**. It assigns near-zero weights ($\alpha \approx 0$) to accidental clicks or irrelevant interactions in the user's history, preventing them from polluting the final user representation.

2.4. Multimodal Learning in Fashion Recommendation

Fashion recommendation is fundamentally a **multimodal** problem. User preference is driven not merely by collaborative signals (interaction history) but significantly by the **visual aesthetics** (e.g., silhouette, texture, color palette) and **semantic attributes** (e.g., brand heritage, material composition, stylistic trends) of the items. Traditional Collaborative Filtering methods, which abstract items into atomic IDs, inherently suffer from the **Cold-Start pathology**: they fail to generate valid representations for strictly new products lacking interaction logs.

To mitigate this, this thesis integrates advanced Visual and Textual representation learning, transitioning from static feature extractors to dynamic **Foundation Models**.

2.4.1. Visual Representation: From CNNs to Prompt-based Foundation Models

The efficacy of a content-aware recommender system is strictly bounded by the quality of its feature extractors.

A. Limitations of Convolutional Neural Networks (CNNs) Seminal works in visual recommendation, such as VBPR [x], relied on pre-trained CNNs (e.g., ResNet-50, VGG-16) initialized on ImageNet. While effective for object classification, these architectures exhibit critical limitations in the fashion domain:

1. **Global Pooling Information Loss:** Standard CNNs typically utilize the output of the final average pooling layer as the item embedding. This operation aggregates spatial information into a single vector, inevitably discarding fine-grained details (e.g., distinguishing an *embroidered* logo from a *printed* one).

2. **Closed-Set Vocabulary:** CNNs are trained to classify a fixed set of categories (e.g., 1000 ImageNet classes). They lack the **open-vocabulary** capability required to understand evolving fashion neologisms (e.g., "gorpcore" or "y2k style") found on social media.

B. The Shift to Florence-2 (Vision-Language Model) To address these shortcomings, this thesis adopts **Florence-2** [x], a unified vision-language foundation model. Unlike task-specific encoders, Florence-2 treats vision tasks as a text-generation problem within an Encoder-Decoder framework.

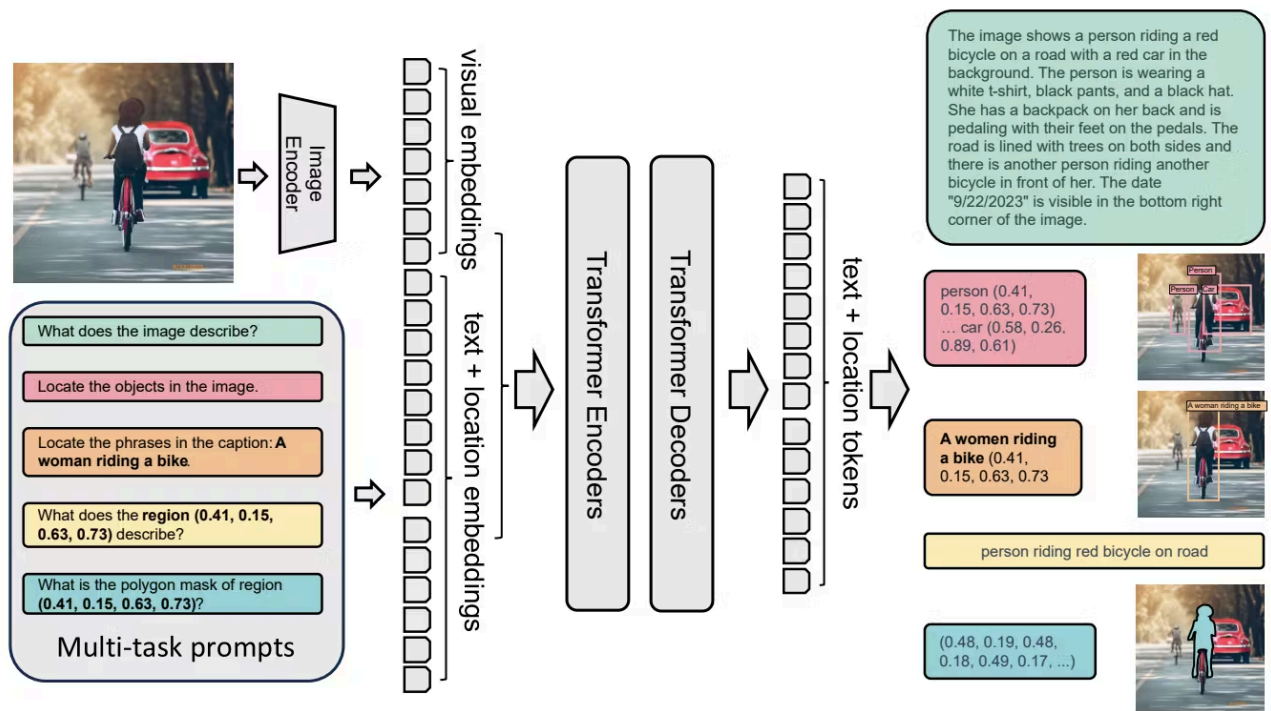


Figure 2.4.1 The unified architecture of Florence-2. The model utilizes a prompt-based encoder-decoder framework to handle diverse vision tasks (e.g., captioning, grounding, segmentation) within a single model. *Source: Xiao et al. [x].*

Architectural Mechanism: Florence-2 utilizes a hierarchical Data-driven Vision Transformer (DaViT) as the image encoder. Unlike standard ViT which processes images as a flat sequence of patches, DaViT maintains a hierarchical structure, capturing both local textures (fabric details) and global structures (outfit silhouette).

Mathematical Formulation: Florence-2 maximizes the likelihood of a textual response $\mathbf{y}$ given an image $\mathbf{I}$ and a specific task prompt $\mathbf{t}$:

$$P(\mathbf{y} | \mathbf{I}, \mathbf{t}) = \prod_{j=1}^L P(y_j | y_{<j}, \mathbf{I}, \mathbf{t}, \theta)$$

Table 2.4.1. Mathematical Notation Used in the Florence-2 Likelihood Model

Notation	Definition
$\mathbf{I}$	Input image for the vision-language model
$\mathbf{t}$	Task prompt specifying the desired operation (e.g., “DETAIL CAPTION”)
$\mathbf{y}=(y_1, y_2, \dots, y_L)$	Sequence of output tokens generated by the model
y_j	The token generated at position j in the output sequence
$y_{<j}$	All previously generated tokens $(y_1, \dots, y_{j-1})$
L	Total length of the output sequence
θ	Learnable parameters of the Florence-2 encoder–decoder model
$P(\mathbf{y} \mathbf{I}, \mathbf{t})$	Conditional probability of the entire text sequence given the image $\mathbf{I}$ and task prompt $\mathbf{t}$
$P(y_j y_{<j}, \mathbf{I}, \mathbf{t}, \theta)$	Probability of generating the next token y_j conditioned on previous tokens and the encoded visual–prompt features.

$\prod_{j=1}^L$	Product operator used to compute joint probability of all output tokens
-----------------	---

Feature Extraction Strategy: For fashion mining, I leverage the **Dense Region Captioning** capability. Instead of a single global vector, the model identifies k regions of interest (RoI) within the outfit. The final visual representation v_t is computed via an attention-weighted pooling of the hidden states corresponding to these regions:

$$v_t = LayerNorm \left(\frac{1}{k} \sum_{i=1}^k Encoder(I)_{region} \right)$$

This ensures the embedding v_t encapsulates dense, fine-grained semantic attributes rather than just a global image label.

Table 2.4.2. Notation for the Region-Level Visual Embedding Formula

Notation	Definition
I	Input image
$Encoder(I)_{region}$	Region embedding extracted from region i of the image I by the encoder
k	Number of detected regions
$\frac{1}{k} \sum_{i=1}^k Encoder(I)_{region}$	Mean pooling of all region embeddings
v_t	Final pooled visual representation for item t
$LayerNorm$	Layer normalization function applied after pooling

2.4.2. Textual Representation: Contextualized Embeddings via BERT

Social media fashion posts are invariably accompanied by unstructured textual metadata, including captions, hashtags, and user comments.

A. Limitations of Static Embeddings Traditional techniques like Word2Vec or TF-IDF generate **static embeddings**, assigning a fixed vector to each word regardless of context. This is problematic for **polysemous** fashion terminology.

- *Example:* The term "*Supreme*" implies a streetwear brand in one context but a qualitative adjective in another. Static models cannot distinguish these nuances.

B. BERT and Masked Language Modeling (MLM) I employ **BERT (Bidirectional Encoder Representations from Transformers)** [x] to generate context-sensitive embeddings. BERT is pre-trained using two objectives: **Masked Language Modeling (MLM)** and Next Sentence Prediction (NSP).

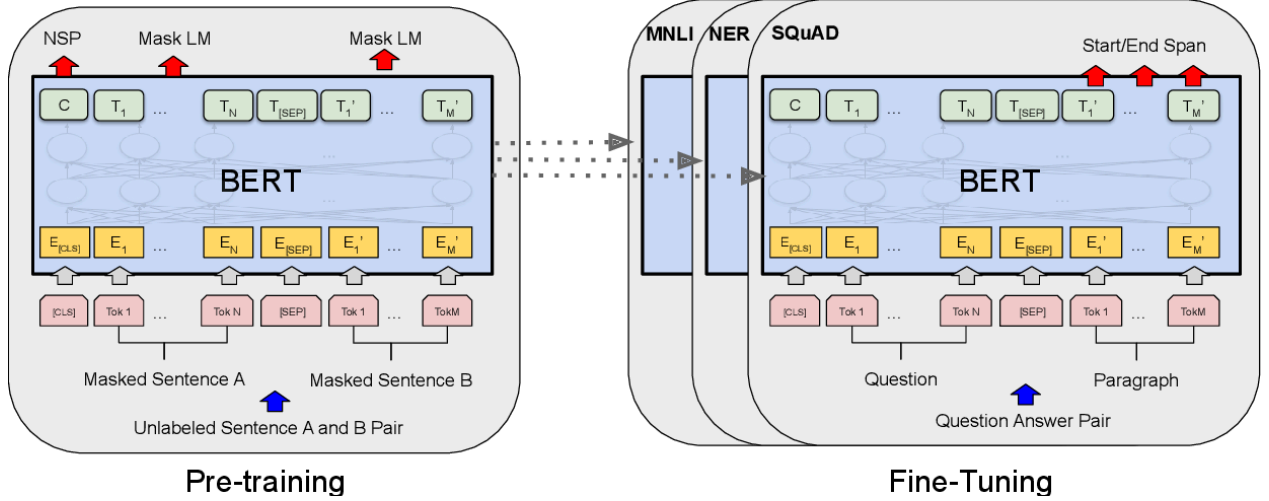


Figure 2.4.2. The architecture of BERT. The model processes a sequence of input tokens and utilizes multi-head self-attention to generate contextualized embeddings. The final textual representation used in this thesis is derived from the output vector C corresponding to the special classification token ($[CLS]$). Source: Devlin et al. [x].

Mathematical Formulation: Given a tokenized input sequence $T = \{t_1, \dots, t_N\}$, BERT constructs a contextual embedding $H^{(l)}$ at layer l using multi-head self-attention mechanisms trained on the **Masked Language Modeling (MLM)** objective:

$$H^{(l)} = \text{TransformerLayer}(H^{(l-1)})$$

The final semantic representation $\mathbf{x}_t^{\text{text}}$ is derived not by averaging all tokens, but by extracting the hidden state of the special classification token ($[CLS]$) from the final layer L , which encapsulates the aggregate semantic meaning of the sequence:

$$\mathbf{x}_t^{\text{text}} = H_L^{[CLS]} \in R^{768}$$

Table 2.4.3. Notation for BERT Transformer Layer and Text Embedding Extraction

Notation	Definition
$H^{(l)}$	Output hidden representation at layer l of the Transformer encoder
$H^{(l-1)}$	Input hidden representation to layer l , i.e., output of the previous layer
<i>TransformerLayer</i> (.)	A single Transformer encoder block consisting of multi-head self-attention, feed-forward network, residual connections, and layer normalization
L	Total number of Transformer encoder layers (e.g., $L=12$ for BERT-base, $L=24$ for BERT-large).
H_L	Output sequence representation from the final (L -th) Transformer layer
$[CLS]$	Special classification token whose contextual embedding is used as the sentence-level representation
x_t^{text}	Final textual embedding for item t , taken from the $[CLS]$ vector of the last layer
R^{768}	Dimensionality of the BERT-base hidden embedding (BERT-large: 1024)

2.4.3. Multimodal Fusion Strategy

To synthesize the heterogeneous information from Florence-2 (v_t) and BERT (x_t^{text}) into a unified item representation x_t for the sequential encoder (BiLSTM), we employ a **Late Fusion** strategy with a non-linear projection.

Fusion Equation: First, the visual and textual vectors are concatenated. Since they typically possess different dimensions ($d_v \neq d_{bert}$), a fully connected (dense) layer is applied to project them into a shared latent space d_{model} :

$$z_{concat} = [v_t \oplus x_t^{text}]$$

$$x_t = GELU(W_{fusion} \bullet z_{concat} + b_{fusion})$$

We utilize the **GELU (Gaussian Error Linear Unit)** activation function instead of ReLU, as GELU provides smoother gradients and is the standard activation used in both BERT and Florence-2, ensuring compatibility across the feature space. The resulting vector x_t serves as the input for the BiLSTM layer described in Section 2.2.

Table 2.4.4. Notation for Multimodal Fusion via Concatenation and Linear Projection

Notation	Definition
v_t	Visual embedding of item ttt, extracted from the visual encoder (e.g., Florence-2)
x_t^{text}	Textual embedding of item ttt, obtained from the BERT [CLS] output.
z_{concat}	Concatenated multimodal feature vector combining visual and textual representations
$[v_t \oplus x_t^{text}]$	Concatenation operator that joins the two vectors along the feature dimension
W_{fusion}	Learnable projection matrix for multimodal fusion
b_{fusion}	Bias term for the fusion layer
$GELU(\cdot)$	Gaussian Error Linear Unit activation, commonly used in Transformer models
x_t	Final fused multimodal embedding for item ttt, used as input to the sequential model (e.g., BiLSTM).

2.5. Contextual Sentiment Analysis

Social media implicit feedback is inherently ambiguous; an interaction does not always imply positive preference (e.g., irony or complaints). Distinguishing valid signals from noise requires robust **Sentiment Analysis (SA)**.

- **Limitations of Lexicon-based Approaches**

Traditional methods like VADER rely on static lexicons. They treat text as a "bag-of-words," often failing to capture context-dependent semantics (e.g., detecting sarcasm or interpreting domain-specific slang like "*sick fit*" as positive).

- **Transformer-based Approach: Twitter-RoBERTa**

To overcome these limitations, this thesis employs **RoBERTa** (Robustly Optimized BERT) [x], which leverages self-attention to generate contextualized embeddings. Specifically, we utilize the Twitter-RoBERTa variant [x], pre-trained on ~58 million tweets.

Mathematical Basis: The sentiment classification is modeled as a probabilistic distribution over classes $Y = \{Negative, Neutral, Positive\}$.

Probability Estimation: Given the input token sequence T , the model outputs logits z , which are normalized via the **Softmax** function:

$$P(y = k \mid T) = \frac{e^{z_k}}{\sum_{j \in \mathcal{Y}} e^{z_j}}$$

Final Classification The discrete sentiment label $\hat{y}$ is determined by selecting the class with the maximum posterior probability:

$$\hat{y} = \arg \max_{y \in \mathcal{Y}} P(y | T)$$

Polarity Score Calculation To quantify sentiment intensity beyond discrete labels, we define a continuous **Polarity Score**. This metric serves as a confidence interval for filtering logic

$$Polarity = P(Positive | T) - P(Negative | T)$$

Range: $[-1, +1]$, where values near -1 indicate strong criticism and +1 indicate strong endorsement.

Table 2.4.5. Mathematical Notations for Contextual Sentiment Analysis

Notation	Definition
C_{ui}	Raw comment text from user u regarding item i
T	Tokenized input sequence fed into the RoBERTa model
z	Logit Vector. The raw output scores from the model's classification head before activation
y	Set of sentiment classes: {0: Negative, 1: Neutral, 2: Positive}
$P(y = k T)$	Posterior Probability. The likelihood that input T belongs to sentiment class k
$\hat{y}$	Predicted Label. The final discrete classification result

2.6. Research Gap and Problem Positioning

Despite the significant advancements in Recommender Systems discussed in the previous sections, applying them to the domain of social media fashion mining presents specific challenges that remain inadequately addressed by current literature. I identify three critical **research gaps** that this thesis aims to bridge.

Gap 1: Coarse-grained Visual Representations in Sequence Models

Most existing sequential recommenders (e.g., GRU4Rec [x], SASRec [x]) focus heavily on optimizing the sequence modeling architecture while treating item features as secondary. Even multimodal variants often rely on outdated, pre-trained CNN backbones like **ResNet-50** or **VGG-16** for visual feature extraction.

- **The Problem:** These CNNs typically utilize **global average pooling**, which compresses the entire image into a single vector. In fashion, this leads to the loss of **fine-grained attributes** (e.g., specific embroidery, collar type, or fabric texture) that are often the decisive factors for user engagement.
- **Our Contribution:** This thesis departs from standard CNNs and investigates the efficacy of **Florence-2**, a Prompt-based Foundation Model. We leverage its **Dense Region Captioning** capability to capture granular aesthetic details that global pooling methods miss.

Gap 2: Inadequacy of Noise Handling Mechanisms

Implicit feedback on social media is inherently noisy. Existing works either ignore this noise or employ **static lexicon-based filters** (e.g., VADER, TextBlob) to remove negative comments.

- **The Problem:** Static lexicons treat text as a "bag-of-words" and fail to detect **context-dependent sentiment** (e.g., sarcasm, domain-specific slang like *"this fit is sick"*). This leads to the rejection of valid training signals or the inclusion of false positives (noise).
- **Our Contribution:** This thesis proposes a **Deep Contextual Denoising** strategy. By deploying **Twitter-RoBERTa** (a Transformer model pre-trained on ~58 million tweets), we leverage self-attention to correctly interpret social media linguistics, ensuring the recommendation model learns exclusively from high-confidence positive signals.

Gap 3: Severe Data Sparsity and Cold-Start

In social media datasets, the user-item interaction matrix is extremely sparse (often >99.9% empty). Traditional Collaborative Filtering (CF) methods fail under these conditions because they rely solely on ID matching.

- **The Problem:** When user interactions are sparse, ID-based embeddings cannot learn meaningful representations for "tail" items or new users (**Cold-Start problem**). Existing hybrid models often fail to effectively transfer knowledge from the visual/textual domain to fill these gaps in the interaction domain.
- **Our Contribution:** This thesis addresses sparsity by implementing a **Multimodal Late Fusion** strategy. Even if an item has *zero* interactions, its rich **Florence-2 visual**

embeddings and **BERT semantic embeddings** allow the model to position it correctly in the latent space relative to similar items, thereby bridging the interaction gap.

Table 2.5: Summary of Research Gaps and Thesis Contributions

Aspect	Existing Approaches (State-of-the-Art)	Limitations (Gaps)	Proposed Thesis Solution
Visual Encoder	CNNs (ResNet, VGG)	Loss of fine-grained details; Closed-set vocabulary	Florence-2 (Prompt-based Dense Captioning)
Noise Filtering	Static Lexicons (VADER)	Fails on sarcasm/slang; Context-agnostic	Twitter-RoBERTa (Context-aware Deep Denoising).
Sequence Modeling	Unidirectional RNN	Ignores future context in offline training	BiLSTM + Attention (Bidirectional context)
Data Sparsity	ID-based Collaborative Filtering	Fails on Cold-Start items.	Multimodal Fusion (Content-driven inference)

CHAPTER 3: METHODOLOGY

3.1. General Idea and Architecture

3.1.1. Design Philosophy: Bridging Semantics and Dynamics

The overarching philosophy of this thesis is that Fashion Recommendation on social media is not merely a collaborative matching problem, but a complex intersection of Visual Understanding and Temporal Sequence Mining. Traditional ID-based approaches (Matrix Factorization) fail to capture the two most critical drivers of user engagement in this domain:

- **Aesthetic Semantics:** Users engage with items because of specific visual attributes (e.g., *a vintage leather texture* or *a bohemian floral pattern*), not because of an arbitrary Item ID.
- **Temporal Evolution:** Fashion preferences are highly volatile and time-dependent. A user's interest shifts dynamically due to seasonal trends or personal style evolution.

To address this, our proposed **Multimodal Sequential Recommendation Framework** is built upon three strategic pillars:

- **Content-First Representation:** Instead of treating new items as "cold" IDs, we leverage Foundation Models (**Florence-2** and **BERT**) to "read" the item's intrinsic content. This allows the system to generalize user preferences to unseen items based on visual/semantic similarity.
- **Noise-Resilient Learning:** Acknowledging the inherent noise in social media feedback, we implement a **Dual-Denoising Strategy**: explicit filtering of negative sentiment using a context-aware **Twitter-RoBERTa** model at the data preprocessing level, and implicit down-weighting of irrelevant interactions via **Multi-Head Attention** at the feature

modeling level. This ensures that the system learns exclusively from high-confidence positive signals.

- **Bidirectional Context Awareness:** We posit that understanding a user's intent at time t requires analyzing the entire session context—both the history that led to the action and the subsequent actions that confirmed it (captured via **BiLSTM**).

3.1.2. Proposed System Architecture

The architecture is engineered as an end-to-end differentiable pipeline consisting of four hierarchical modules, as illustrated in **Figure 3.1**.



3.2. Data Collection and Dataset Description

To evaluate the proposed Multimodal Sequential Framework, a domain-specific dataset was curated from **Instagram**, focusing specifically on the fashion and apparel domain. The data

acquisition pipeline involved crawling public posts related to clothing trends, extracting associated metadata, and capturing user engagement via comments.

3.2.1. Data Acquisition Process

The data collection process was executed via a custom web scraping pipeline designed to target clothing-related hashtags, captions (e.g., *#killian*, *#fashiontrends*, *#streetwear*). The raw data was initially extracted in **JSON (JavaScript Object Notation)** format to preserve the hierarchical structure of nested objects such as user profiles and comment threads.

To facilitate efficient data manipulation and subsequent feature extraction, the raw unstructured JSON logs were parsed and flattened into two distinct relational tables stored in **CSV (Comma-Separated Values)** format:

- **Post Data:** Capturing content-centric information (Captions, Hashtags, Images).
- **Comment Data:** Capturing interaction-centric information (User feedback)

3.2.2. Post Data Specification

The **Post Data** repository serves as the primary source for content features. Each record corresponds to a unique Instagram post.

	post_id	timestamp	postUser	caption	hashtags	likesCount	commentsCount	image
0	1	2023-06-02 16:34:43+00:00	maryleest	Cannes 2023 with @kilianparis ❤️ #KilianCannes ...	KilianCannes	32070	142	https://scontent-ord5-1.cdninstagram.com/v/t51...
1	2	2023-01-01 07:49:16+00:00	tinaabeysekara	As the clock struck midnight to ring in 2022, ...	NaN	6565	86	https://instagram.fyei4-1.fna.fbcdn.net/v/t51...
2	3	2023-05-29 18:57:51+00:00	maryleest	The famous stairs ❤️ Photo @gustave_durin Dre...	NaN	28936	123	https://instagram.fpeg1-2.fna.fbcdn.net/v/t51...
3	4	2023-03-07 19:30:42+00:00	stephaniebroek	I visualized this moment so many times before....	CHANELFallWinter	4764	215	https://scontent-lhr8-1.cdninstagram.com/v/t51...
4	5	2023-05-23 21:11:12+00:00	maryleest	Got to witness such historical moment in cinem...	CannesFilmFestival	13379	123	https://instagram.fprg5-1.fna.fbcdn.net/v/t51...

Figure 3.2.1. Post DataFrame

This is a brief description of the data:

Table 3.2.1. Description of the Post dataframe

Column name	Data Types	Description	Role in model
post_id	int64	Unique identifier for each post	Linkage Key
timestamp	object	Date and time when the post was created	Temporal Ordering
postUser	object	Username of the post creator	Metadata
caption	object	Text content of the post	Input for BERT
hashtags	object	Tags associated with the post	Input for BERT
likesCount	int64	Number of likes on the post	Global Popularity
commentsCount	int64	Number of comments on the post	Popularity Metric
image	object	URL of the post's image	Input for Florence-2

3.2.3. Comment Data Specification

The **Comment Data** repository captures the user-item interaction signals. In the context of this thesis, user comments serve as **implicit feedback**, reflecting user interest in specific fashion items.

	post_id	ownerUsername	timestamp	comments	likes
0	1	wilsonjunior5055	2023-06-05 12:27:24+00:00	I have a proposal for u, send me a DM please!	0.0
1	1	mariavmh	2023-06-04 16:46:40+00:00	Woaaaah	0.0
2	1	mariavmh	2023-06-04 16:46:34+00:00	:smiling_face_with_heart-eyes::smiling_face_wi...	0.0
3	1	stylemesoftly_	2023-06-04 15:44:54+00:00	:red_heart:red_heart:red_heart:	0.0
4	1	jillviv	2023-06-04 15:06:03+00:00	:smiling_face_with_heart-eyes::smiling_face_wi...	0.0

Figure 3.2.2. Comment DataFrame

Table 3.2.2. Description of the Comment dataframe

Column name	Data Types	Description	Role in model
post_id	int64	Unique identifier for each post	Linkage Key
ownerUsername	object	Username of the commenter	User ID (for Sequence)
timestamp	object	Date and time when the comment was posted	Sequence Sorting
comments	object	Text content of the comment	Input for Twitter-RoBERTa (Filtering)
likes	float	Number of likes on the comment	Quality Weighting

3.2.4. Data Linkage and Interaction Formation

To construct the sequential training data, the **Post Data** and **Comment Data** are merged using a relational join operation on the **post_id** key.

Mathematically, let $\mathcal{P}$ represent the set of posts and $\mathcal{C}$ represent the set of comments. The unified dataset $\mathbf{D}$ is formed such that:

$$D = \{(u, i, t, c) \mid \exists p \in \mathcal{P}, k \in \mathcal{C} : k.post_id = p.post_id\}$$

This integrated structure allows the system to associate specific **User interactions** with rich **Item content** (Caption, Hashtag, Image), enabling the extraction of sequential patterns essential for the BiLSTM model.

3.3. Data Cleaning, Preprocessing and Exploratory Data Analysis (EDA)

3.3.1. Data Cleaning and Preprocessing

A. Post DataFrame:

- **Data shape:** 827 rows and 8 columns (827,8)
- **Drop unnecessary columns:** Drop the ‘image’ and ‘hashtags’ columns, since we download the first image of each post and the hashtags are already included in the caption column, from which we can extract them.
- **Data type conservation:** The timestamp fields in the dataset were converted from string (object) to datetime format using **pandas library** in Python. This enables temporal analysis, such as identifying posting or commenting trends over time.

Table 3.3.1. Data Type of each column after conservation

Column name	Data Types
post_id	int64
timestamp	Datetime64[ns, UTC]
postUser	object
captions	object
likesCount	int64
commentsCount	int64

- **Missing value:**

Handle missing value in caption column by “**no_caption**”.

```

post_id      0
timestamp    0
postUser     0
caption      7
hashtags    353
likesCount   0
commentsCount 0
dtype: int64

```

```

post_id      0
timestamp    0
postUser     0
caption      0
likesCount   0
commentsCount 0
dtype: int64

```

Figure 3.3.2. Missing value before and after handling in Post dataframe

- **Caption Processing:** Instagram captions contain a mixture of plain text, hashtags, mentions, URLs, and emojis. We developed a multi-step pipeline to extract and clean these components:

- **Stage 1: Hashtag extraction**

Hashtags prefixed with “#” are extracted using regular expression pattern matching, converted to lowercase, and deduplicated to serve as categorical labels for content filtering. Posts without hashtags are assigned a placeholder value “**no_hashtag**”.

- **Stage 2: Mention extraction**

Mentions prefixed with “@” are extracted to identify referenced brands, designers, and collaborators associated with each fashion post. Similar to hashtags, mentions are normalized to lowercase and deduplicated. Posts without mentions are assigned a placeholder value “**no_mention**”.

- **Stage 3: Text cleaning**

- + Remove URLs
- + Strip hashtags and mentions
- + Convert to lowercase
- + Remove punctuation

- + Eliminate emojis through Unicode pattern matching
 - + Normalizing whitespace
- This produces clean natural language text suitable for embedding generation.

post_id	timestamp	postUser	caption	hashtags	likesCount	commentsCount	mentions
1	2023-06-02 16:34:43+00:00	maryleest	cannes 2023 with wearing dress	[kiliancannes]	32070	142	[kilianparis, mrselfportrait]
2	2023-01-01 07:49:16+00:00	tinaabeysekara	as the clock struck midnight to ring in 2022 i...	[no_hashtag]	6565	86	[thetreasurededit]
3	2023-05-29 18:57:51+00:00	maryleest	the famous stairs photo dress jewelry hair makeup	[no_hashtag]	28936	123	[gustave_durin, marmarhalim, yessayan, balmmain...
4	2023-03-07 19:30:42+00:00	stephaniebroek	i visualized this moment so many times before ...	[chanelfallwinter]	4764	215	[chanelofficial]
5	2023-05-23 21:11:12+00:00	maryleest	got to witness such historical moment in cinem...	[cannesfilmfestival]	13379	123	[no_mention]

Figure 3.3.3. Post DataFrame after preprocessing

B. Comment DataFrame

- **Data shape:** 5432 rows and 5 columns (5432, 5)
- **Data type conservation:** Timestamp fields in the dataset were transformed from string (object) to datetime format, in alignment with the post dataframe.

Table 3.3.1. Data Types after conservation in Comment DataFrame

Column name	Data Types
post_id	int64
commentUser	Datetime64[ns, UTC]
timestamp	object
comments	object
likes	float

- **Missing value:** Handle missing values in columns: drop rows without **commentUser** and replace missing comments with “**no_comment**”

post_id	0	post_id	0
commentUser	71	commentUser	0
timestamp	71	timestamp	0
comments	100	comments	0
likes	71	likes	0
dtype: int64		dtype: int64	

Figure 3.3.3 Missing value before and after handling in Comment dataframe

- **Comment processing:** Instagram comments contain informal language, emojis, mentions, and varying sentiment expressions ranging from positive engagement to spam or criticism. We developed a multi-step pipeline to clean the text and analyze user sentiment for quality filtering:

- **Stage 1: Text Normalization and Cleaning**

Before sentiment analysis, raw comments undergo a cleaning process to remove artifacts while preserving emotive signals.

- + **Artifact Removal:** We utilize Regular Expressions (Regex) to strip URLs, HTML tags, and specific user mentions (@username), as these tokens do not contribute to the sentiment polarity of the item itself.
- + **Emoji Retention Strategy:** Unlike aggressive preprocessing pipelines that strip special characters, we explicitly **retain emojis that have been converted into textual representations**. In the fashion domain, these symbols (e.g., :fire:, :heart:, :dress:) act as high-intensity sentiment amplifiers. The Twitter-RoBERTa tokenizer can process these

textual tokens, treating them as semantic units equivalent to words, thereby preserving the emotional signal embedded in the input data.

- + **Repeated Character Normalization:** Social media text often contains elongated words for emphasis (e.g., "*soooo goood*"). We normalize these to their canonical forms to ensure dictionary matching.

- **Stage 2: Contextual Sentiment Computation (Twitter-RoBERTa)**

We employ the **Twitter-RoBERTa** model (cardiffnlp/twitter-roberta-base-sentiment), a transformer-based language model fine-tuned on approximately 58 million tweets [x]. This model is specifically designed for social media text, making it well-suited for analyzing Instagram comments which share similar linguistic characteristics with Twitter posts.

- **Stage 3: Negative Noise Filtering**

To ensure the recommendation model learns from constructive signals, we implement a strict filtering logic based on the predicted sentiment labels.

Filtering Logic: Interactions classified as **Negative** (e.g., criticism, spam, or dissatisfaction) are identified as noise and removed from the dataset. We retain only **Neutral** and **Positive** comments, positing that these represent valid user attention and endorsement.

$$M_{valid} = \begin{cases} 1 & \text{if } \arg \max P(y|C) \in \{\text{Neutral, Positive}\} \\ 0 & \text{if } \arg \max P(y|C) = \text{Negative} \end{cases}$$

Outcome: Applying this filter resulted in the retention of **5,197 comments**, representing **96.9%** of the original scraped dataset. This high retention rate indicates that the majority of user

interactions in the collected sample were constructive, with only a small fraction of explicit noise requiring removal.

	post_id	timestamp	clean_comment	sentiment	polarity
0	1	2023-06-05 12:27:24+00:00	i have a proposal for u send me a dm please	neutral	0.2074
2	1	2023-06-04 16:46:34+00:00	smiling_face_with_heart eyes smiling_face_with...	positive	0.8857
3	1	2023-06-04 15:44:54+00:00	red_heart red_heart red_heart	neutral	0.2744
4	1	2023-06-04 15:06:03+00:00	smiling_face_with_heart eyes smiling_face_with...	positive	0.9805
5	1	2023-06-04 08:39:28+00:00	fire fire fire	neutral	0.0278

Figure 3.3.4. Positive Comments After Filtering Process

- Stage 4: User Interaction Sequence Construction

The final preprocessing step transforms the filtered interactions into temporal sequences suitable for the BiLSTM architecture.

- + **Chronological Sorting:** Valid interactions are grouped by unique User IDs. For each user u , the interactions are sorted chronologically based on the timestamp t to form an ordered sequence:

$$S_u = [(i_1, t_1), (i_2, t_2), \dots, (i_k, t_k)] \text{ where } t_1 < t_2 < \dots < t_k$$

- + **K-Core Filtering (Sequence Length Constraint):** To ensure sufficient historical context for sequential learning, we apply a **3-core filtering** strategy. Users with fewer than 3 interactions ($|S_u| < 3$) are excluded, as extremely short sequences do not provide adequate patterns for the model to capture temporal dynamics.

Final Dataset Statistics: After sequence construction and length filtering, the final dataset consists of **231 unique users** with valid, sentiment-verified interaction sequences, ready for sequential recommendation training.

	commentUser	interaction_sequence	posts_sequence	sequence_length
30	92thestudio	[{'post_id': 188, 'timestamp': 2023-05-17 14:5...]	[188, 216, 91, 57]	4
105	_surisingh	[{'post_id': 369, 'timestamp': 2023-05-18 09:0...]	[369, 361, 360]	3
109	_vanessa.riedl	[{'post_id': 246, 'timestamp': 2023-05-01 13:0...]	[246, 88, 57]	3
113	_yycc99	[{'post_id': 69, 'timestamp': 2023-03-08 20:40...]	[69, 64, 17]	3
144	adrianericmorales	[{'post_id': 795, 'timestamp': 2023-02-23 11:5...]	[795, 797, 792]	3

Figure 3.3.5. Comment Data After Preprocessing

C. Image Data Retrieved to Generate Descriptions

Images were downloaded directly from the URLs in the original dataset and standardized to a uniform format. Each image was saved using its corresponding **post_id** as the filename (e.g., **1.png** for **post_id = 1**), ensuring a consistent mapping between textual and visual data. After preprocessing, a total of **822 images** were successfully collected, with each image corresponding to a unique post. This one-to-one alignment provides a reliable foundation for subsequent description generation and multimodal analysis.

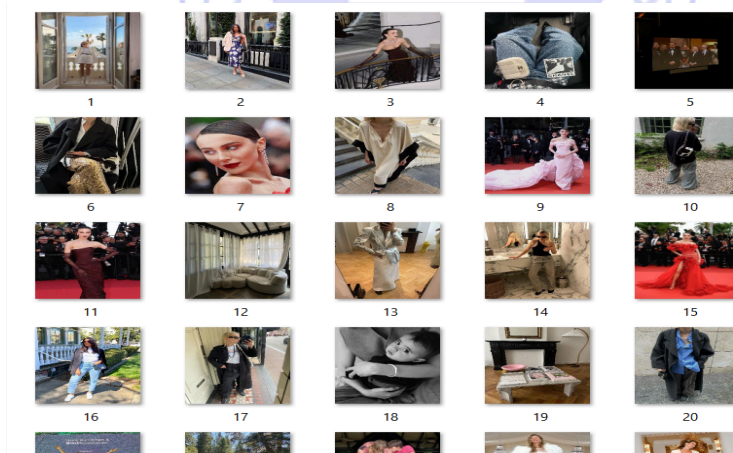


Figure 3.3.6. Sample of Downloaded Images from the url

3.3.2. Exploratory Data Analysis

Comprehensive Exploratory Data Analysis (EDA) was conducted to uncover the underlying characteristics of the dataset, identify interaction patterns, and detect potential anomalies before modeling.

A. Engagement Distribution and Interaction Patterns

We analyzed the distribution of **likesCount** and **commentsCount** to assess the popularity dynamics of fashion posts.

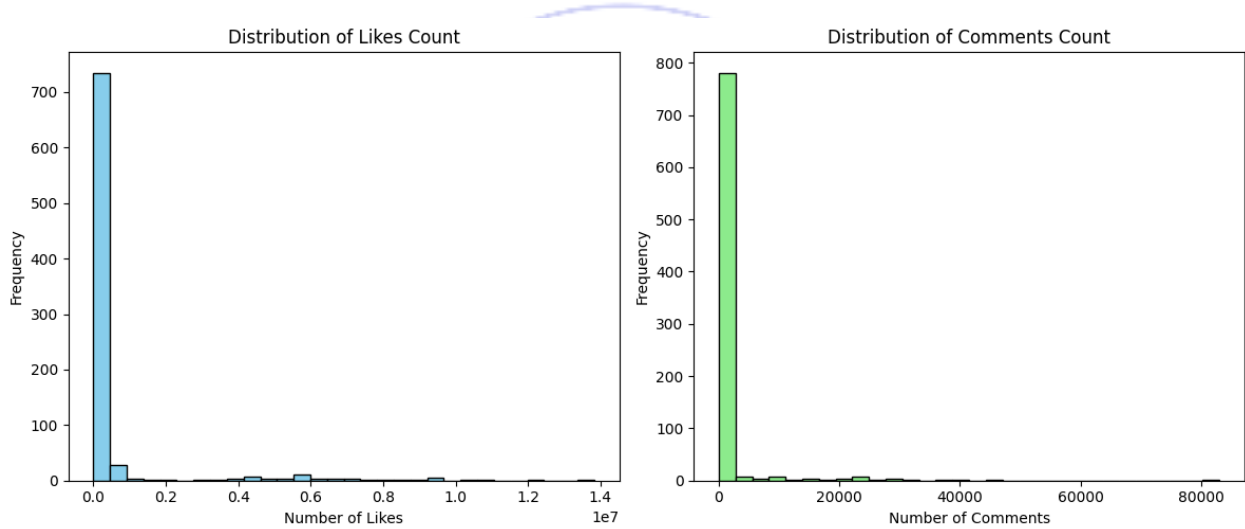


Figure 3.3.7. Likes and Comments Distribution

As illustrated in **Figure 3.3.7**, the engagement metrics exhibit a heavy **Right-Skewed Distribution (Long-Tail)**.

- **Observation:** The majority of posts receive moderate engagement, while a small fraction of "viral" posts garner exceptionally high interaction counts.

- **Correlation:** There is a moderate positive correlation (**Pearson = 0.61**) between likes and comments, indicating that visual appeal (likes) often translates into discussion (comments).
- **Implication for Modeling:** This extreme sparsity and popularity bias pose a challenge for traditional ID-based recommenders, validating our choice to incorporate **Content-Based Features** (Florence-2/BERT) to improve recommendations for the "tail" items (less popular posts).

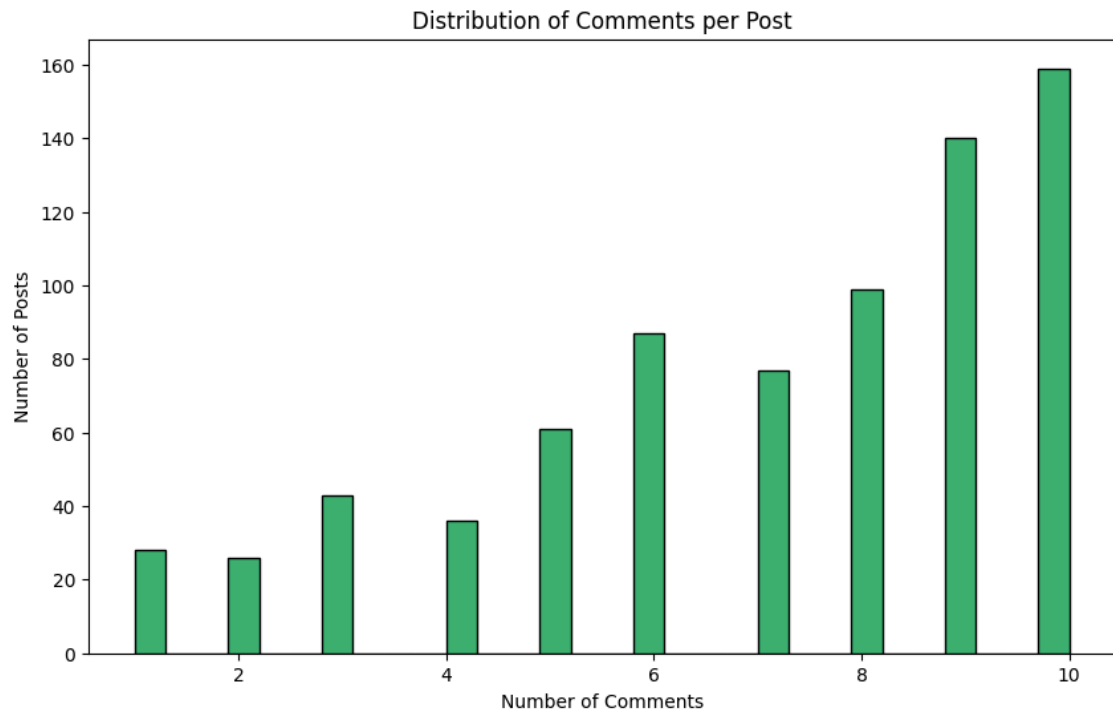


Figure 3.3.8. Distribution of Comments per Post

Similarly, the distribution of comments per post (**Figure 3.3.8**) confirms that user interaction is sparse, with most users engaging with only a select few items.

B. Temporal Dynamics Analysis

To understand user behavior over time, we visualized posting frequency and response latency.

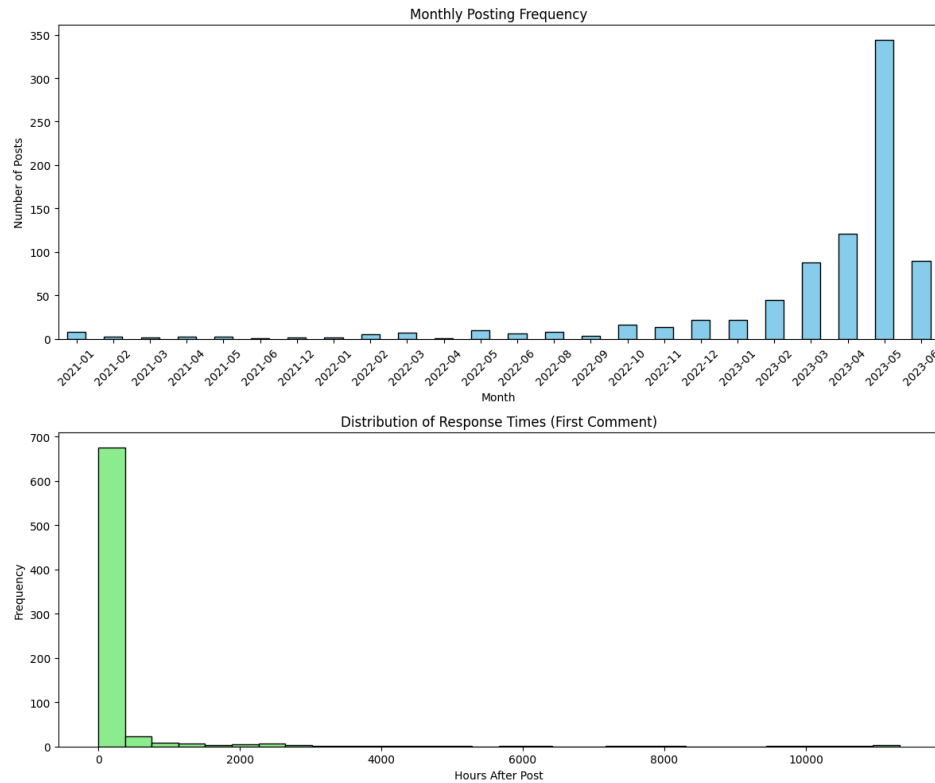


Figure 3.3.9. Posting Frequency by Month

- **Seasonality:** Figure 3.3.9 reveals distinct seasonal trends in posting frequency, likely driven by fashion cycles (e.g., Summer/Winter collections).
- **Response Latency:** Figure 3.3.9 indicates the time lag between a post's creation and its first comment. This temporal signal is crucial for the Sequential Model (BiLSTM) to learn the "freshness" and immediate relevance of fashion trends.

C. Semantic and Sentiment Analysis

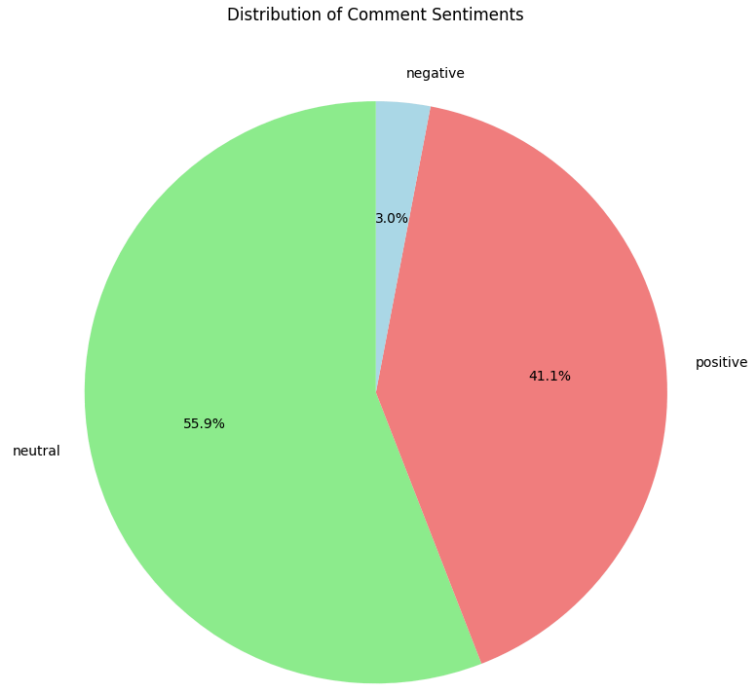


Figure 3.3.11. Distribution of Comment Sentiments

Insight: As shown in **Figure 3.3.11**, while the majority of comments are Neutral or Positive, a notable segment exhibits Negative sentiment. This empirical evidence validates the necessity of the **Twitter-RoBERTa filtering pipeline** (described in **Section 3.3.1**) to remove these noise signals, preventing the recommendation engine from learning false preferences.

3.4. Visual Feature Extraction (Florence-2)

3.4.1. Overview

In multimodal fashion recommendation systems, visual information plays a crucial role in understanding user preferences and item characteristics. Unlike traditional collaborative filtering approaches that rely solely on user-item interactions, our methodology incorporates rich visual

features extracted from fashion images to capture fine-grained style attributes such as colors, patterns, silhouettes, and overall aesthetics.

This section presents our approach to visual feature extraction using **Florence-2**, a state-of-the-art vision-language foundation model. The extracted visual features are subsequently integrated with textual features to form comprehensive multimodal representations for each fashion post.

3.4.2. Florence-2 Vision-Language Model

- **Model Selection Rationale**

We selected Microsoft's Florence-2 as our visual feature extractor based on several key considerations:

- **Unified Vision-Language Architecture:** Florence-2 employs a sequence-to-sequence framework that naturally bridges visual and textual modalities, making it well-suited for generating semantically rich descriptions of fashion images.
- **Pre-training on Diverse Visual Tasks:** The model was pre-trained on FLD-5B, a dataset containing 5.4 billion annotations across 126 million images, enabling robust zero-shot performance on fashion imagery without domain-specific fine-tuning.
- **Detailed Caption Generation:** Unlike classification-based approaches that output discrete labels, Florence-2 generates natural language descriptions that capture nuanced visual attributes essential for fashion understanding.

- **Architecture:**

The Florence-2 model operates on a unified sequence-to-sequence framework, comprising two primary architectural components: a hierarchical vision encoder and a multimodal transformer:

- **Vision Encoder: Dual Attention Vision Transformer (DaViT)** The backbone of the visual system is the **DaViT** architecture. Unlike standard Vision Transformers (ViT) that perform computationally expensive global self-attention, DaViT efficiently captures both local and global context through two orthogonal attention mechanisms:
 - + **Spatial Window Attention:** Captures fine-grained local features (e.g., texture, patterns).
 - + **Channel Group Attention:** Captures global semantic relationships and abstractions.

Mathematical Formulation: Given a raw input image $I \in \mathbb{R}^{H \times W \times 3}$, the DaViT encoder transforms it into a sequence of flattened visual tokens V . This process preserves the hierarchical semantic structure necessary for dense captioning:

$$V = \Phi_{DaViT}(I) \in \mathbb{R}^{N_v \times d}$$

where Φ_{DaViT} represents the stacked dual-attention layers

N_v is the number of visual tokens (dependent on the feature map size)

d is the hidden dimension of the embedding space

- **Multimodal Encoder-Decoder (Seq2Seq Generation)** To generate semantic descriptions, the model integrates visual features with textual instructions. The architecture treats the task as a conditional text generation problem.

The visual tokens V and the task-specific prompt embeddings P (e.g., corresponding to the token "<DENSE_REGION_CAPTION>") are fused and processed. The Decoder generates the textual description sequence $T = \{t_1, t_2, \dots, t_L\}$ auto-regressively:

$$P(T | V, P) = \prod_{i=1}^L P(t_i | t_{<i}, \text{Encoder}(V), P)$$

Here, the probability of the next token t_i is conditioned on the previously generated tokens $t_{<i}$, the encoded visual representations, and the task prompt. This formulation allows the model to "translate" visual pixel data into structured fashion attributes.

The overall architecture is illustrated in **Figure 3.4.1**.

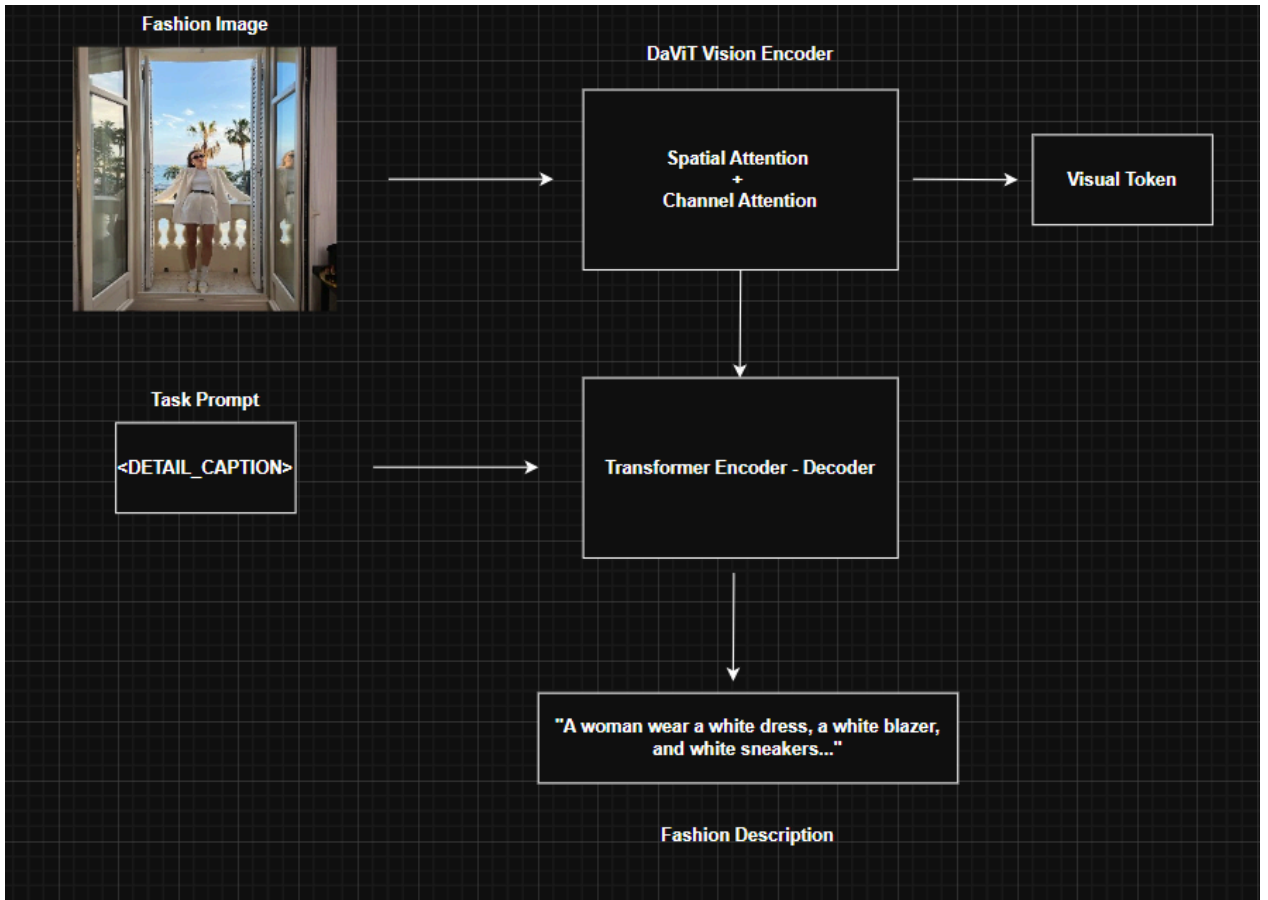


Figure 3.4.1. Florence-2 Architecture

3.4.3. Implementation Strategy: Task and Decoding

To adapt this general-purpose model for fashion recommendation, we defined specific task configurations and decoding strategies.

A. Prompt Selection

Table 3.4.1. Task Types and Characteristics

Task Type	Description	Average Length
CAPTION	Brief, single-sentence description	~20 tokens
DETAIL_CAPTION	Standard description with key attributes	~50 tokens
MORE_DETAIL_CAPTION	Comprehensive description with fine-grained details	+100 tokens

For the fashion recommendation system, we selected **MORE_DETAIL_CAPTION** as our primary task. This choice is motivated by the need to capture comprehensive visual information including:

- **Clothing categories:** dresses, blazers, jeans, accessories ...
- **Color attributes:** primary colors, color combinations, patterns ...
- **Style characteristics:** formal, casual, elegant, sporty ...
- **Material textures:** silk, denim, leather, cotton ...
- **Contextual elements:** occasion, setting, styling choices ...



Figure 3.4.2. Sample outputs generated by Florence-2 using the
<MORE_DETAILED_CAPTION> prompt

B. Generation Configuration

To ensure high-quality and reproducible feature extraction, the decoding process utilizes **Beam Search** with the following hyperparameters:

- **num_beams=3**: Balances exploration and exploitation to reduce hallucination compared to greedy search
- **max_new_tokens=512**: Provides sufficient capacity for describing complex outfits without truncation
- **do_sample=False**: Enforces deterministic generation, ensuring that the same image always yields the same description vector.

3.4.4. Feature Encoding and Semantic Alignment

A distinct advantage of our approach is the use of natural language as an intermediate representation layer. Instead of confronting the heterogeneity gap between raw pixels and semantic tags directly, we translate the visual signal into a dense textual description. This facilitates Semantic Alignment, allowing visual features to be mapped into the same linguistic latent space as the metadata (captions, hashtags).

Encoding Mechanism: Let τ_{gen} be the detailed description sequence generated by Florence-2 for image I (as configured in Section 3.4.3). To convert this variable-length sequence into a fixed-size vector suitable for the sequential model, we utilize the **BERT-base** encoder.

Mathematical Formulation: The description τ_{gen} is tokenized and fed into BERT. We extract the hidden state of the special classification token ([CLS]) from the final transformer layer:

$$\mathbf{v}_{\text{visual}} = \text{BERT}_{[\text{CLS}]}(\text{Florence2}(I, \text{prompt})) \in \mathbb{R}^{768}$$

Where $\mathbf{v}_{\text{visual}}$ is visual embedding vector

$\text{Florence2}(I, \text{prompt})$ is model generating text from image and prompt

Output Interpretation: The resulting vector $\mathbf{v}_{\text{visual}}$ possesses two critical properties:

- **Visual Fidelity:** It encapsulates the fine-grained visual attributes (texture, cut, color) identified by Florence-2
- **Semantic Compatibility:** Since it shares the same vector space (768 dimensions) as the textual features extracted, it facilitates efficient multimodal fusion without requiring complex manifold alignment techniques

3.4.5. Processing Pipeline

The complete visual feature extraction pipeline is illustrated in **Figure 3.4.2**.

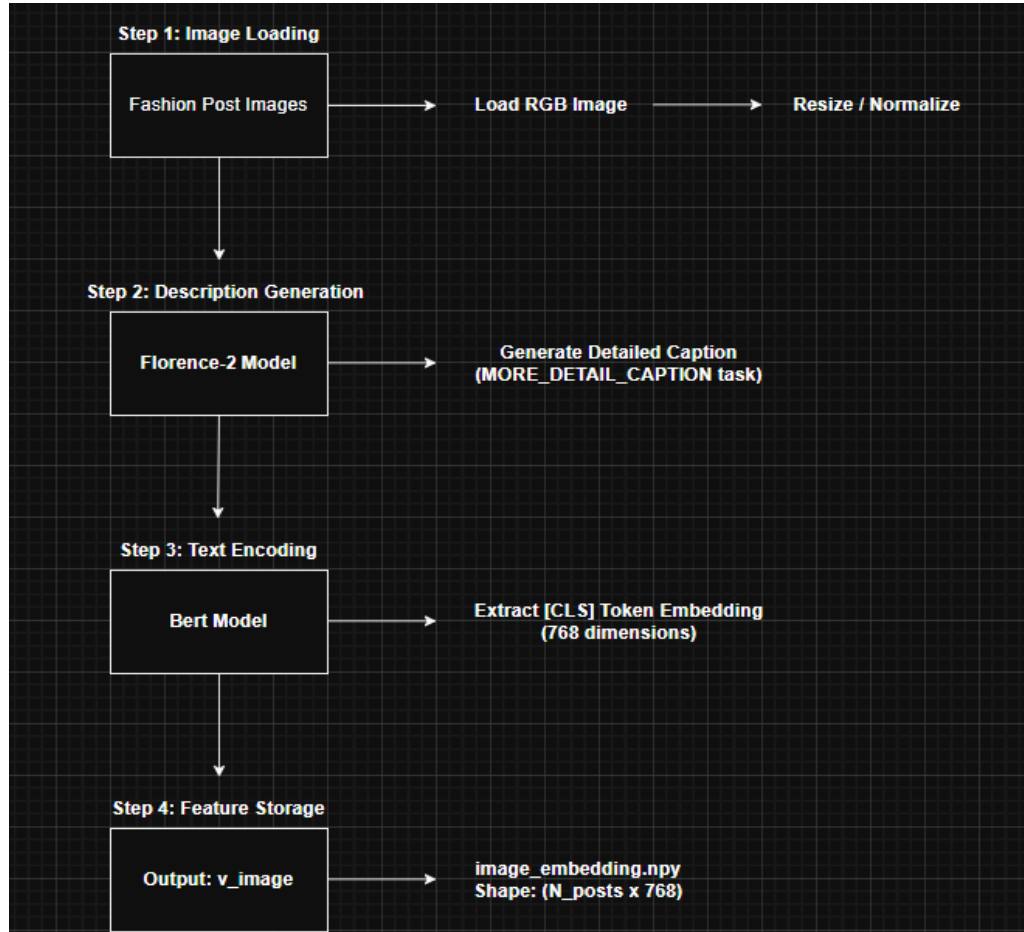


Figure 3.4.3. Visual Feature Extraction Pipeline

3.4.6. Implementation Statistics

To validate the robustness and efficiency of the proposed visual extraction pipeline, we executed the process on the collected Instagram dataset. The implementation metrics are summarized in **Table 3.4.2.**

Table 3.4.2. Visual Feature Extraction Statistics

Metric	Value	Technical Implication
Total Images Processed	822	Full coverage of the curated dataset
Successful Conversions	817 (99.4%)	High robustness of Florence-2 to varied social media

		image formats and aspect ratios
Failures	5 (0.6%)	Due to corrupt headers or non-fashion content (e.g., text-only images) which were subsequently discarded.
Avg. Description Length	89.3 tokens	Indicates high Semantic Density . For comparison, standard COCO captions average ~10-15 tokens. This confirms the model is capturing deep details.
Output Dimension	768	Perfectly aligned with BERT-base hidden size.
Processing Time	~16.4 minutes	Feasible for offline preprocessing on NVIDIA GTX 3050Ti

Conclusion on Visual Module: The high success rate (99.4%) and the richness of the generated descriptions (avg. 89.3 tokens) confirm that **Florence-2** is an effective backbone for overcoming the limitations of global pooling CNNs. The resulting embeddings v_{visual} provide a dense, attribute-aware foundation for the downstream BiLSTM recommendation model.

3.5. Textual Feature Extraction (BERT)

3.5.1. Overview

Social media fashion posts contain rich textual information that complements visual content. In the context of Instagram fashion influencers, textual data encompasses three distinct modalities: captions, hashtags, and mentions. Each modality carries unique semantic signals that contribute to understanding user preferences and content characteristics.

This section presents our methodology for extracting dense vector representations from textual data using **BERT** (Bidirectional Encoder Representations from Transformers). The extracted textual features are subsequently combined with visual features to form comprehensive multimodal embeddings.

3.6. Implementation platform and libraries

The preprocessing, sequence mining, model training, and clustering tasks for the fashion recommendation system were implemented in Jupyter Notebooks using Python (version 3.9+). This environment facilitated interactive development, data visualization, and iterative experimentation. The following libraries and tools were utilized to ensure efficient preprocessing, feature extraction, model training, and user clustering.

All tasks were performed in Jupyter Notebooks, leveraging Python's robust data science ecosystem for preprocessing, exploratory data analysis (EDA), sequence mining, and model training (Bi-LSTM). The interactive nature of Jupyter Notebooks enabled seamless integration of code, visualizations, and documentation.

Key libraries included:

- **Pandas and NumPy:** For data manipulation and numerical operations.
- **scikit-learn:** For TF-IDF keyword extraction.
- **NLTK and spaCy:** For tokenization, lemmatization, and stopword removal.
- **emoji:** For emoji normalization.
- **VADER:** For sentiment analysis
- **Matplotlib, Seaborn, and WordCloud:** For visualizations.
- **transformers (Hugging Face):** For BERT embeddings used in Bi-LSTM.
- **Prefixspan:** For sequence mining to identify frequent user interaction patterns

- **TensorFlow or PyTorch:** For implementing the Bi-LSTM model to capture sequential dependencies in user preferences.
- **json and requests:** For JSON parsing and URL validation.



CHAPTER 5: CONCLUSIONS

This study successfully implemented a fashion recommendation system using sequence mining and Bi-LSTM, leveraging the Instagram dataset to personalize recommendations based on user interactions and content. The Sequence Mining module, using PrefixSpan, identified frequent interaction patterns, enhancing user clustering and Bi-LSTM inputs. The Bi-LSTM model effectively captured sequential and semantic dependencies, achieving a promising top-5 accuracy of 64%. K-Means clustering grouped users with similar fashion preferences, with a silhouette score of 0.61, supporting targeted recommendations.

Contributions: Developed an integrated pipeline combining preprocessing, feature engineering, sequence mining, clustering, and Bi-LSTM for fashion recommendations. Demonstrated the effectiveness of PrefixSpan for identifying user interaction patterns and Bi-LSTM for sequential modeling. Achieved meaningful user clustering with a similarity threshold of 0.5, enhancing personalization.

Limitations: The dataset is limited to public Instagram posts, potentially missing private interactions. Bi-LSTM training is computationally intensive, and sparse data impacts top-K accuracy. Image data (image URLs) was not analyzed, limiting visual feature integration.

Future improvement: Expanding the dataset to include private posts or additional platforms (e.g., Pinterest). Incorporating image analysis using convolutional neural networks (CNNs) to leverage image data. Optimizing Bi-LSTM hyperparameters and exploring attention

mechanisms to improve accuracy. Testing alternative clustering algorithms (e.g., DBSCAN) to enhance user grouping

This study has established a strong foundation for building scalable and practical recommendation systems. By leveraging clustering techniques, it improves the interpretability and relevance of recommendations, providing a promising path for further development and real-world applications.



REFERENCES

- [1] Schuster, M., & Paliwal, K. K. (1997). *In Bidirectional Recurrent Neural Networks*. IEEE Transactions on Signal Processing, 45(11), 2673–2681.
- [2] Agrawal, R., & Srikant, R. (1995). *In Mining Sequential Patterns*. Proceedings of the Eleventh International Conference on Data Engineering, 3–14.
- [3] Liu, B., Hu, X., & Chen, H. (2018). *In Fashion Recommendation with Side Information via Deep Learning*. IEEE Access, 6, 1918–1927.
- [4] Albalawi, R., Othman, S. S. B., Zain, A. M., Mubarak, A., Siraj, M., Eltayeb, M., Hassan, Z. B., & Algarni, A. (2023). *Data mining-based recommendation system using social networks—an analytical study*. Heliyon, 9(3), e13455.
- [5] Wu, Q., Gao, C., Gao, Y., Chen, P., & Cheng, H. (2020). *Deep Bi-LSTM Networks for Sequential Recommendation*. Entropy, 22(8), 870
- [6] Kang et al., "Self-Attentive Sequential Recommendation," *arXiv preprint arXiv:1904.06690*, 2019.
- [7] Nguyen et al., "Multimodal Deep Learning for Comment Generation," *arXiv preprint arXiv:2302.03883*, 2023.
- [8] Agrawal & Srikant, "Mining Sequential Patterns," *Proceedings of the Eleventh International Conference on Data Engineering*, 1995.

- [9] Sutskever, I., Vinyals, O., & Le, Q. V. (2014). Sequence-to-sequence learning with neural networks. *NeurIPS*.
- [10] Zhang, C., Xu, R., & Li, Y. (2020). Hybrid pattern-based and neural generation for comment recommendation. *WWW*.

